Silesian
University
of Technology

FINAL PROJECT

Measurement module for three-phase electrical infrastructure

Mateusz Józef WOŁOWCZYK
Student identification number: 293729

Programme: Control, Electronic, and Information Engineering
Specialisation: Informatics

SUPERVISOR
dr inż. Krzysztof Tokarz
DEPARTMENT Graphics, Computer Vision and Digital Systems
Faculty of Automatic Control, Electronics and Computer Science

Gliwice 2024

Thesis title

Measurement module for three-phase electrical infrastructure

Abstract

Nowadays electrical energy became a crucial resource, especially in the industry. Industrial energy meter's have greater requirements than the household ones. They have to take into account higher voltage (up to 400 [V] line-to-line in Europe), as well higher current, not one but three phases and measurement of reactive energy, as companies must pay for it transfer. With the development of Internet of Things such meters are more often connect to networks, which enhances data reading, storage and processing. OpenThread is a new communication protocol created primarily for IoT devices. It is an open-source implementation of Thread released by Google.
This thesis goal is to design and build a electrical energy measuring module, which can communicated with other devices and the entire Internet thru OpenThread. The key part is the introduction to this protocol's functionality and basic concepts of electrical energy measurement. It describes the project's requirements, helpful tool and development process. Afterwards a test is performed with real industrial machine. At last some potential improvements are discussed.

Keywords

electrical energy measurement, Internet of Things, Zephyr RTOS, Thread, OpenThread

Tytuł pracy

Moduł pomiarowy energii elektrycznej w instalacji trójfazowej

Streszczenie

W dzisiejszych czasach energia elektryczna stała kluczowym zasobem, zwłaszcza w przemyśle. Przemysłowe mierniki energi mają większe wymagania niż domowe. Muszą brać pod uwagę większe napięcie (do 400 [V] międzyfazowego w Europie), również większy prąd, nie jedna a trzy fazy oraz pomiar energi bierniej, gdyż firmy muszą płacić jej przesył. Wraz z rozwojem Internetu Rzeczy takie mierniki są coraz cześciej podłączane do sieci, która poprawia odczyt, przechowywanie oraz przetwarzanie danych. OpenThread jest nowym protokołem komunikacyjnym stworzonym przede wszystkim dla urządzeń Internetu Rzeczy. Jest to otwarto źródłowa implementacja Thread wydana przez Google.
Celem tej pracy jest zaprojektowanie oraz budowa modułu pomiarowego energii elektrycznej, który komunikuje się z innymi urządzeniami i całym Internetem przy użyciu

OpenThread. Główną częścią jest wprowadzenie do funkcjonalności tego protokołu oraz podstawowych pojęć związanych z pomiarem energii elektrycznej. Opisuje wymagania projektu, przydatne narzędzia jak również proces jego tworzenia. Następnie wykonany jest test z użyciem rzeczywistej maszyny przemysłowej. Na koniec omawia kilka potencjalnych usprawnień.

Słowa kluczowe

pomiar energii elektrycznej, Internet Rzeczy, Zephyr RTOS, Thread, OpenThread

Contents

Chapter 1

Introduction

Nowadays electrical energy has become a important resource. It is used practically all of the day. For personal use mostly to assure comfort. The industrial production could not exist without it. The electrification and automation had an groundbreaking impact of the humanity. The energy consumption is basically kept on track for bill preparation. Additionally it can be used to inform about the power saving properties of particular devices and equipment.

The Industry 4.0 or sometimes called smart factory is an ongoing industrial revolution. It's main part is the Machine-to-Machine communication, which can be driven by the Internet of Things or more specific Industrial Internet of Things. The concept of IoT is a seamless connection between sensors and actuators to improve and optimize production processes. The metering module described in this thesis is an example of sensor. It's main purpose is to gather data about consumed energy and assure a data access with a wireless connection.

The goal of the thesis is to design and construct a electrical energy measuring module. It has to operate in three-phase installations. It must enable the measurement of active, reactive and apparent energy. The module shall use OpenThread protocol for communication. It shall has a DIN rail enclosure.

Firstly the thesis deals with designing the hardware architecture with it's core components: microcontroller from the nRF52 Series with OpenThread support from Nordic Semiconductor, electrically erasable programmable read-only memory, real time clock, input / output expanders and metering integrated circuits from Analog Devices. Due to the stock lack of three-phase specific ICs, the system uses three separate one-phase metering ICs. Voltage and current transformers are used as input circuits to theme. The last hardware piece is a simple power supply unit. Then on the firmware side, the project's uses the nRF Connect Software Development Kit with Zephyr, a small real-time operating system. It provides support for OT functionality and drivers for all hardware components. The EEPROM is used with a direct operating system application programming interface, while the rest is implemented basing on bus drivers for Inter-Integrated Circuit and Serial

Peripheral Interface. OpenThread enables easy device connection to already existing mesh network, service registration protocol assigns discoverable service name to host address and port, domain name system address resolving for accessing any server on the Internet by humane readable name, time update from server based on the simple network time protocol and user datagram protocol for exchanging data between the module and a client device. Lastly the software for client (computer) communication with metering module is developed in form of a command line application. It allows to read measured values.

The Problem analysis chapter introduces the definition of energy and three-phase installations, as well as some existing solutions. It describes the OpenThread protocol too. The Requirements and tools chapter describes the project's requirements, tools needed or helpful during the design process and implementation, as well as the process itself. The External specification chapter contains the module's installation procedure, manual of the entire system and usage scenarios. The Internal specification chapter presents the concept and system architecture of the hardware part with corresponding firmware code. It also shows the important libraries, data structures and functions from OpenThread library. The Verification and validation chapter is about testing, bugs and results of the created solution. The last chapter Conclusions discusses the obtained outcome with regard to objectives and requirements, possible improvements, encountered difficulties and problems.

This thesis is a introduction to OpenThread protocol, as not many complex guides are available besides the published in OT project web site [23]. It presents how to start a network, connect a device and use functionalities like SRP, IPv6, DNS, SNTP, UDP. It also presents a custom printed circuit board design, fitting into a prefabricated DIN rail enclosure, for three separate metering units with current and voltage transducers.

Chapter 2

Problem analysis

2.1 Energy in electrical circuits

The most basic physical definition of energy is shown in the formula 2.1.

$$Energy = Power * Time \tag{2.1}$$

In simple word it's the product of power and time, when it was used. For example, if a electric kettle has a nominal power of 2400 watts and boils the water for 5 minutes, the energy consumed by this devices can be calculated, basing on formula 2.1, as follows:

$$E = 2400[W] * \frac{1}{12}[h] = 200[Wh] \tag{2.2}$$

It can be observed that the mostly used unit of electrical energy is watt hour $[Wh]$, while the SI unit is joule $[J]$.
The formula 2.3 is a generalization of formula 2.1. It is more suitable, when the power of the examined object varies over time.

$$Energy = \int Power(t)dt \tag{2.3}$$

Based on the above formulas the problem of energy measurement is further divided into two sub problems of power and time measurement. In case of embedded system, sometimes called real-time systems, the issue of time measurement is not complex. A microcontroller or real time clock counts oscillations from a quartz resonator with constant frequency, which are recalculated into time lapse.

When speaking about energy in electrical circuits, firstly they have to be split into direct and alternating current. In the of the DC circuits, the power is calculated as in equation 2.4, where U is the voltage and I is the current. All the equations listed below

are taken from [49].

$$P = UI \tag{2.4}$$

In case of AC circuits the power calculation is not so effortless, because the voltage $u(t)$ (formula 2.5) and current $i(t)$ (formula 2.6) are sinusoidal waveforms:

$$u(t) = U_m \sin \omega t \tag{2.5}$$

$$i(t) = I_m \sin (\omega t + \phi) \tag{2.6}$$

where U_m is voltage amplitude, I_m is current amplitude, ω is the pulsation, t is an arbitrary moment, ϕ is the phase shift angle between current and voltage.
The later used U and I are respectively root mean square values of voltage and current.

Hence the additional elements, in the AC systems are present three types of power therefore three types of energy, not one as in DC.

2.1.1 Active power

The active power (calculated with formula 2.7) is present in systems with resistive loads. It is the most meaningful kind of power. In energy receivers, it is the useful piece remolded into other kind of energy; heat energy in heaters; light energy in lighting; mechanical energy in engines. It is measured by a watt-meter and it's unit is watt [W].

$$P = UI \cos \phi \tag{2.7}$$

2.1.2 Reactive power

The reactive power (calculated with formula 2.8) is present in systems with reactant loads, such as coils and capacitors. It has no purpose. It just flows from the source to the receiver and back. Such behavior unnecessarily overloads the power grid. In industrial machines it is required to minimize it's presence. It can be achieved with balancing both types of reactance, inductive and capacitive. It is measured by a var-meter and it's unit is var [VAR].

$$Q = UI \sin \phi \tag{2.8}$$

2.1.3 Apparent power

The apparent power (calculated with formula 2.9) is present in systems with impedance loads. In fact all of the real systems have impedance, because of the elements imperfections. Even a simple cable has a real model with resistor, coil and capacitor. It is measured either directly by sometimes found volt-ampere-meter or indirectly by voltmeter and ammeter. It's unit is volt-ampere [VA].

$$S = UI \tag{2.9}$$

The relationship between all kinds of power (energy) is presented by the formula 2.10. It is often called the power triangle.

$$S^2 = P^2 + Q^2 \tag{2.10}$$

All three types of power have different units, although all of them are equal to $1[\frac{J}{s}]$. They are used only to distinguish their source.

2.2 Three-phase electrical infrastructure

The three-phase electrical infrastructure is particular case of the multi-phase electrical infrastructure, which is defined in [7] as " multi-phase system is called a set of electrical circuits in which sinusoidally alternating source voltages of equal frequency, shifted in phase with each other and produced in a single energy source, called a multi-phase generator or alternator". In case of the three-phase system the phase shift is equal to 60 [°] or $\frac{2\pi}{3}$ in radians.
The three-phase circuit may be symmetrical or not; use three or four wires (L1 - phase 1, L2 - phase 2, L3 - phase 3, N - neutral). "The phases of the alternator and the impedance of the load can be associated in a delta or Wye pattern" [7]. The patterns may be mixed, e.g. a delta alternator may be connected to Wye load.

It also states how to handle the power measurement and the following energy measurement in such infrastructure. There are described many solutions to this problem, however the most universal is a by using three watt meters (one for each phase) and sum up the separate values for the total power. It applies not only to the active power, but likewise reactive and apparent power, with respective meters. This method is not the simplest one, but ensures reliability as the system parameters may be not consistent.

Fortunately, there are available many integrated circuits from well known chipmakers, like Analog Devices [9], STMicroelectronics [66] and Microchip [67]. They perform all the

necessary calculations based on voltage, current signal and clock oscillations from quartz resonator. The calculated energy values are stored in registers and accessed through an interface.

2.3 OpenThread

OpenThread [23] is an open-source implementation of Thread [36] released by Google. It is supported by ARM Holdings, NXP, Apple, Nest Labs, OSRAM, Samsung, Silicon Labs, Somfy, Tyco International, Qualcomm, Amazon, Yale, Eve, Aqara and Nanoleaf. The specification defines "an IPv6-based reliable, secure, and low-power wireless device-to-device communication protocol for home and commercial building applications". OpenThread is composed of the following networking layers: IPv6, IPv6 over Low-Power Wireless Personal Area Networks, IEEE 802.15.4 with MAC security, Mesh Link Establishment, Mesh Routing. It is created to have a narrow platform abstraction layer and many configuration options to minimize the memory size, which makes OT easy portable.
The 802.15.4 (2.4 [GHz] frequency band) radio is used not only by OpenThread, but also Zigbee, WirelessHART. There are other 802.15 mesh networking protocols, such as Z-Wave, and Bluetooth Low Energy.

Thread was designed to include specific primary features [34]:

- "simple installation, start up and operation",
- "authentication of all devices in network and encrypted communication",
- "self-healing mesh network, with no single point of failure and spread-spectrum techniques to provide immunity to interference",
- "low-power devices sleep to reduce energy consumption and may operate on one battery for years",
- up to 16385 devices all of roles in one mesh network.

2.3.1 Mesh network

The term of "mesh network" was quote before, but was not clearly defined. It is a structure, where all the nodes are connected directly, dynamically and non-hierarchically to all other available nodes and cooperate together to efficiently route data within the network. The dynamical self-organization ensures reliability, as fault nodes can be easily removed from the structure with no harm to it.
OpenThread does not fulfil all above characteristics, more particularly the non-hierarchically and all nodes connection. The reasons are explained in the next subsection.

2.3.2 Node Roles and Types

One of the advantages of OpenThread in comparison to Wi-Fi and Bluetooth Low Energy is the differentiation of devices, in the mesh network named nodes, with respect to roles and types [26].
The specification defines four roles:

- Leader - a Router that manages all other Routers. It is dynamically self-elected. A Thread network can have only one Leader
- Router - forwards packets in network and it is enabled all time
- End Device - connected with only one Router, does not forward packets and may be turned off to reduce energy usage
- Border Router - described in detail in chapter 3. A Thread network can have multiple Border Routers

and two categories of types:

- Full Thread Device - always on; knows all multicast and IPv6 addresses mapping; can operate as a Router (Parent) or an End Device (Child):
  - Router
  - Router Eligible End Device - can be promoted to a Router
  - Full End Device - cannot be promoted to a Router
- Minimal Thread Device - communicates only with it's Parent; can only operate as an End Device (Child):
  - Minimal End Device - always on and receives messages
  - Sleepy End Device - normally off. Occasionally poll Parent for messages

The transition between roles is dynamically and depends on the network structure. Every device starts as a End Device. If it is a REED and the only device, it becomes the Leader; if there is already a Leader it becomes a Router.

2.3.3 Microcontroller architecture

OpenThread supports two microcontroller architecture designs: System-on-Chip and Co-Processor (Radio Co-Processor, Network Co-Processor) [24].

The SoC design is the most typical, what can be expected from an embedded system. The microcontroller is the main component and handles all of the operation performed by the entire system.

The RCP and NCP approaches are quite different. They are not the main unit, but only and addition as their names suggest. There must be a host processor, which is responsible for the system operation. They are controlled through a serial interface with Spinel protocol.
In the RCP design, the host processor maintains the OpenThread core, while the co-processor acts only as a minimal medium access controller. It is used by the OpenThread Border Router and can work together with OpenThread Daemon - ot-daemon to get access to a Thread network via a standard macOS or Linux computer. In general it is desired for devices with more powerful processors and less sensitive to power constraints, which ensures the reliability of the network.
On the other hand in the NCP design, the co-processor handles all of the Thread features and host processor is responsible for the application layer. The separation of the higher-power host and lower-power OpenThread benefits in improved power control, as well as independent development and testing of this two parts. It is usable for gateway devices or appliances with other purposes.
Spinel is a general management protocol for communication and management of the co-processor by the host. Created firstly for Thread-based co-processors, however with layered approach it is adaptable to other network technologies [56]. It is included with OpenThread and has a Python CLI tool called Pyspinel.

2.4 Existing solutions

As far as the author of this project is aware no three-phase energy meters with OpenThread have been developed yet.

In case of connection technology only a simple smart plug can be found [11]. It is limited to one-phase and should be considered for a simple home usage. With a maximum power of 2500 [W] it may insufficient even for hairdryers and electric ovens, nevertheless it's mobile application has interesting features like schedules and projected energy cost.

There are similar solutions based on the Matter over Wi-Fi standard. First is made mainly for DC power measurement [45]. Second is more advanced with AC support [48]. They should considered more like prototypes than finished products. The designs are not very sophisticated, rather based on simple electronic elements. Both are using the Nordic Semiconductor nRF7002 Development Kit, which in theory could be replaced with Thread compatible units produced by the same manufacturer.

The idea of a three-phase energy meter device is not well covered in the literature. The discussed resource [68], [2] and [46] picture in general the identical approache; a input

circuit from power grid, metering unit and peripheral, such as display or wire / wireless connection modules controlled with a microcontroller.

The commercial solutions like Fluke [12] is a very advanced metering device prepared mostly for diagnosis purposes, nonetheless it complies with the electrical requirements and has support for Wi-Fi and Bluetooth with a mobile companion app.
The products from IAMMETER [43] and Shelly [35] are more suitable, with the desired form factor, electrical parameters and extensive IoT technologies support (of course besides OpenThread). They are the closest meters compared to the goal of this thesis.

NXP Semiconductor offers an entire Three-Phase Power Meter Reference Design [64]. The recommended microcontroller supports Thread protocol. Enables additional measurements of power factor and frequency. It is capable to measure current up to 100 [A] and voltage in range from 85 to 264 [V].

Microchip offers a similar design guide [38]. They overall concept is not much different. The guide is quite old, from 2008, and outdated. Newer solutions shall be taken in consideration. Communication with the meter is available only thru USB.

The Power Meter Reference Design [10] from Analog Devices and articles [15] [16] from "Elektronika Praktyczna" present a very similar approach, however they are based on IC types. They are sigle-phase meters, nonetheless described in detail. The current channel use transformers, while for voltage a direct connection. Both have linear power supplies, displays and programming interfaces. They differ in peripherals, the first has an EEPROM, while the second an RTC and keypad.

The concepts from the last three paragraphs are a good base for the electronic design of this project.

Chapter 3

Requirements and tools

3.1 Requirements

Project's requirements were stated at the initial part of the development and served as constraints for all design choices. They were divided into two parts: functional and nonfunctional.

3.1.1 Functional requirements

1. measuring module shall operate in 3-phase installations,
2. must enable the measurement of active, reactive and apparent energy,
3. should have a DIN rail enclosure,
4. shall use OpenThread protocol,
5. shall be powered directly from the power grid without any adapter,
6. client device shall communicate with the module, while connected only to standard Wi-Fi / Ethernet network.

3.1.2 Nonfunctional requirements

1. presence of high voltage and current shall be limited,
2. number of external connections shall be reduced to minimum,
3. installation process shall be simple, only one Flathead screwdriver needed,
4. device can be added to a network with use of simple credentials,
5. user's software shall be easy to use, based on straightforward commands,
6. calibration parameters shall be calculated mostly by the module itself.

3.2 Tools

The development of the measuring module required a variety of tools from different areas of engineering. They are described in detail below and divided into software and hardware tools, with two special placed between those categories, the border router and development kit.

3.2.1 Software tools

The software tools were utilized mainly during the designing and programming phases.

EAGLE

EAGLE is an electronic design automation application, dedicated for printed circuit boards. The app consist of two parts: drawing a schematic diagram; placing all elements on the board and routing all necessary connections. For a few years this application has been replaced by Autodesk Fusion 360, which is more complex (additional 3D modeling feature), while the author of this thesis has some experience with EAGLE and it still fulfills it's objective, it was chosen as PCB design software. Afterwards the production files were sent to China and manufactured as at the present time is the most common and cheapest way of ordering PCBs.

Visual Studio Code

Visual Studio Code is a popular open-source code editor. It is recommended by Nordic Semiconductor for nRF Connect Software Development Kit. It is also used as the integrated development environment for Python.

nRF Connect

nRF Connect is available for Visual Studio Code and Desktop.
The VSC version is an extension. It modifies the VSC to become an complex IDE, with integrated basic functions: build, debug, flash, serial port terminal and real-time terminal. It adds also a graphical editors for devicetree overlay files and Kconfig configuration files. Device overlay [51] defines or redefines hardware elements, like pin configuration, bus configuration and auxiliary integrated circuits. Kconfig [50] provides the configuration flags for software support, like GPIO, networking, OpenThread and logging.
The Desktop version is a set of applications, which serves 3 main goals represented by seperate apps. Toolchain manager is the essential app to start developing with NCS. It installs all the necessary components and tools needed by the SDK, such as Zephyr SDK, CMake, west and ninja. The next two goals are specific to the nRF52840 Dongle, which cannot be recognized by VSC, because of the lack of SEGGER J-Link debug probe [57].

First is the programmer to program the built code to the microcontroller. Second is the serial terminal, which allows to communicate with the programmed dongle.

Python

Python is a popular programming language. It is not a very operation optimal language, but it is easy to create applications with it. It has a very expanded package library, which reduce the amount of work from software developer to introduces new functionality in comparison to the C programming language. However the SDK for microcontroller and OpenThread library are limited to C and C++, so Python can used only for the command line application.

Git

Git is a very helpful tool during software development. It is a source control tool. It tracks the changes of files added to a repository, based on commits of current files state. Git works locally, but it's data can be uploaded to remote servers such as GitHub, GitLab or BitBucket to share progress and backup the project. Graphical user interface is available in Visual Studio Code. It was created for development of the Linux kernel and remains the most popular tool of it's kind.

3.2.2 Hardware tools

The hardware tools were used for two main purposes: assembling of the measurement module and it's succeeding verification.

Soldering station

During the assembly the soldering station is the most important tool. It solders electronic elements to the printed circuit board.

Magnifier with backlight

To fit the PCB with all elements in the desired enclosure, majority of the electronic elements are in the form of surface mount device rather than through-hole technology. In addition many ICs are available only as SMDs. The magnifier with backlight is a labor-saver in case of soldering those elements and checking solder quality, by improving the visibility.

Multimeter

Multimeter is the most elementary measuring instrument. As the name suggest it can measure multiple quantities, like voltage, current, resistance and capacitance. During the

development of this project, it was used as a voltmeter to check the correctness of supply voltage values and continuity tester to validate the connections between soldered elements.

Oscilloscope

Oscilloscope allows to observe the voltage signals with high frequencies. It is very useful for verifying the operation of I2C and SPI buses. It was mainly used to analyze the behavior of voltage level translator, which had some malfunctions described in detail in chapter 6.

Voltage tester

Voltage tester is the most basic tool, while working with high voltage. It's main and only task is to sense the presence of voltage. Typically it is found in the form of a screwdriver, which may be useful during installation, with a light indicator. It's a helpful tool for reducing the risk of electric shock.

3.2.3 OpenThread Border Router

The OpenThread Border Router [27] is a crucial element of the Thread infrastructure. It acts like a intermediary between the OT network and standard Wi-Fi / Ethernet network. It allows for the client device to communicate with the metering module with no additional effort.

OTBR has some additional features, as described in the guide introduction, utilized by this thesis [27]:

- "web GUI for configuration and management",
- "bidirectional service discovery via mDNS (Wi-Fi/Ethernet) and SRP (Thread network)",
- "NAT64 for connecting to IPv4 networks",
- "DNS64 to allow Thread devices to initiate communications by name to an IPv4 server".

The last two cases are considered, since some internet service providers limit their offer to IPv4 connection to customers and the IPv6-only OT communication can fail, while trying to gain access to the wide area network, even the local area network router has no problem with IPv6.

The OpenThread Border Router used in this project was created using the official guide without any modifications. It was based on BeagleBone Black single board computer (Raspberry Pi was not chosen, because of the ongoing electronics shortage) and a nRF52840 Dongle (it requires an additional compilation switch for the radio co-processor built: −DOT_BOOTLOADER=USB).

The web graphical user interface has a few nice to have functions, which eliminates the need of using command line interface of BeagleBone computer.

- Join tab shows available Thread networks and allows to join them if the credentials are known.
- Form tab is used to from a Thread network, with default or proprietary parameters, like network name, network key and channel. Once formed the OT network is restarted with every SBC restart.
- Status tab presents the current state of border router. Beside the settings from the form tab, the IPv6 address, OpenThread version, RCP state and version can be verified.
- Commission tab allows to add a new device to the network with use of the Joiner Credential - a device-specific string of all uppercase alphanumeric characters, for example J01NME.
- Topology tab exemplifies the network topology with all devices types: leader, router and child devices. They are identified by the last 16 bits of the Routing Locator [25].

3.2.4 nRF52840 Development Kit

The development kit produced by Nordic Semiconductor [61] has the same microcontroller as the dongle used in the final design. It's main features are the real-time terminal, which handles the logging messages and a virtual communication port to control and verify the OpenThread instance operation thru a command line interface. It is really helpful compared to the dongle, when developing the firmware.

3.3 Methodology of design and implementation

The methodology was not clearly defined for the entire project.
The hardware part used a waterfall-like method. Electronic components were chosen starting from the most essential to the least significant. Next the necessary additional circuitry was defined. Lastly all elements were incorporated into PCB design.

The software was implemented differently, more in the agile way. Code was written in small fragments. Each change was tested and tracked with Git right after. The development started with an OpenThread proof of concept. Later basic support for all ICs was made. The majority of features were added depending on the contemporary needs, without a fixed plan. It is also true for command line application, which was developed in parallel to OT and metering ICs firmware.

3.3.1 Electronics

The first electronic element chosen for the project was the metering integrated circuit, as the main component. Analog Devices was the manufacturer with the largest choice, however due to the electronics shortage only one IC type was available - ADE7753. It features the measurement of active, reactive, and apparent energy. Unfortunately it can measure just one phase. Thankfully the producer considered such case, that three single-phase ADE ICs can be used for a three-phase application [44].
Next those ICs must be connected to the power grid. There are a few options for the current channel: low resistance shunt, current transformer, Hall Effect sensor and air-core current transformer. The current transformer is the most available and safe option to chose. For the voltage channel, there are two possibilities: voltage transformer and direct connection. The transformer isolates the module from the high voltage grid, so it was selected as the destined solution. Both channels need an additional attenuation network for two reasons. Higher frequency noise has to be filtered out and input differential signals levels cannot exceed ± 0.5 [V] at normal operation.

The second most important component was the microcontroller. OpenThread is supported by many vendors, such as Infineon, Nordic Semiconductor, NXP, Silicon Labs, Texas Instruments, Samsung and STMicroelectronics.
The Arduino Nano 33 BLE was an interesting choice. It is based on the supported nRF52840 SoC from Nordic. However for now it is not directly supported by OT nor the Arduino framework is compatible with OT library. It can be flashed with an OT binary, but the serial port communication could not be established.
Dongle is a suitable form for the MCU. It simplifies the PCB design, programming and testing. Other dongles are offered by Nordic directly and NXP. Among those the selected nRF52840 Dongle is the most available and cost efficient solution. In such case only the System-on-Chip architecture design is possible.

Basing on the reference power meter application note from Analog Devices [10], an EEPROM is added to store measured values with a time stamp got from the RTC.

At the end the additional components were chosen. The dongle has a limited number of GPIO pins, so not all of the ICs connections can be handled. An I/O expander fixed this issue. The SoC operates at 3.3 [V] logic and the ADE7753 needs 5 [V] logic, a voltage level translator is necessary to not damage any pins and assure proper operation of the metering ICs.

Fortunately the designed PCB with all of the elements fitted in the bought DIN rail case and there was no need to design a dedicated one nor divide the system into separate parts.

3.3.2 Programming

The programming part began with choosing the framework, which supports both the OpenThread library and nRF52840 microcontroller. The OT developing guide [17] presents a possible solution. Unfortunately the platform hardware abstraction is quite complicated and requires a lot of low level programming.
On the other hand the nRF Connect SDK with Zephyr RTOS [63] is the simplest way to prepare the firmware code. Nordic Semiconductor made an introductory course [59], which explains how use generic GPIO application programming interface, devicetree, logging and I2C serial communication.

EEPROM is handled by the respective Zephyr SDK API, so not much programming was needed. RTC uses mainly the I2C bus functions for register read / write operations and `time.h` header file for time conversion. I/O expander is based on a GPIO driver for the similar IC TCA6424A already included in the Zephyr SDK. Metering IC requires only functions for the SPI bus.

The default OpenThread instance is operated by the Zephyr network library. Additionally all the files from OT library are present and can used as described in API reference [28]. There is no power constraints to the system, so the router eligible end device type has been selected.

Lastly, to finish the firmware code, the overlay file and some hardware support fragments had to be rewritten to utilize the pins available in the dongle form factor as it was developed with the dev kit.

The command line application is a simple Python script. It's most important advantage is the `zeroconf` package, which enables the mDNS discovery. More precisely it finds

the measuring module IPv6 address and respective port number, by using just the service name. The socket handles UDP packets transfer and **struct** package transforms the received raw bytes to floating-point numbers representing the measured values.

Chapter 4

External specification

4.1 Installation procedure

4.1.1 Electrical infrastructure

The measuring module may be placed on DIN rail. It has in total 14 connections. They are divided into two groups: power sense and power supply (in figure 4.1 they are visible respectively on the left and right side). At this moment, there is no possibility to power the module from sense inputs. If the wiring is already prepared (e.g. the module is a replacement for another measuring unit) only one Flathead screwdriver is needed to install the device.
There are three sets of power sense inputs, one for each phase. First two are for voltage. In the Wye configuration one terminal is phase, while the other is neutral. In the delta configuration both are phases (it is important to remind that there must be a 60 [°] phase shift between them). Second two are for current transformer. It must to spooled on a wire, in which the current flows to or from measured circuit. The schematic diagram of Wye and delta configurations are illustrated respectively in figures 4.2 and 4.3.
There are two power supply connectors: one for 230 [V] phase wire and the other for the neutral wire. The device should be turned on after the commissioning is started on a border router.

4.1.2 Client device

To run the command line application a Python installation is required (at least version 3.9.6) and an additional library for mDNS support. Python is system dependent. The zeroconf module may be installed with 4.4.
It is important to notice that a Thread border router with commissioner functionality is required, as well as the used computer has to be connect to that router either thru Wi-Fi or Ethernet. The Pre-Shared Key for the Joiner is "M3T3R1T".

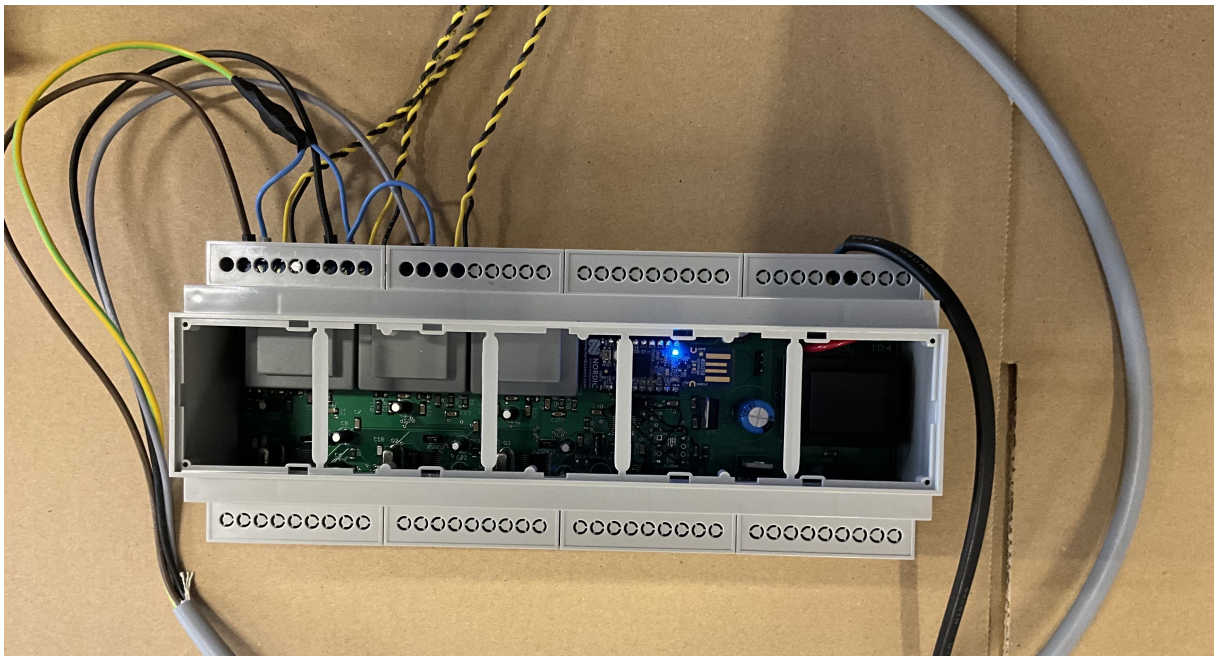

Figure 4.1: The measuring module side in enclosure.

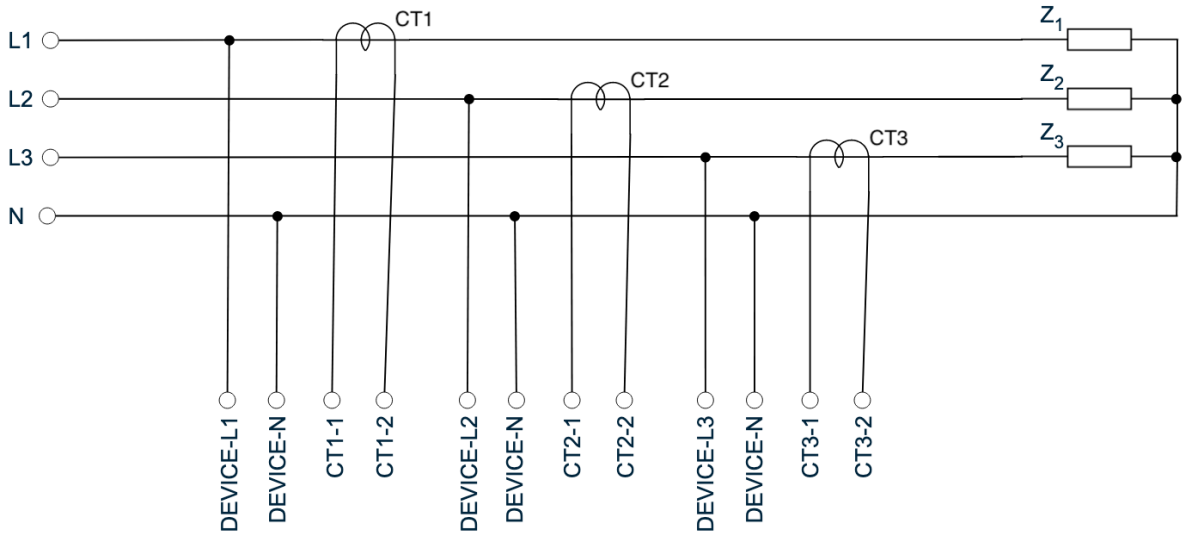

Figure 4.2: Schematic diagram of device connection in Wye configuration.

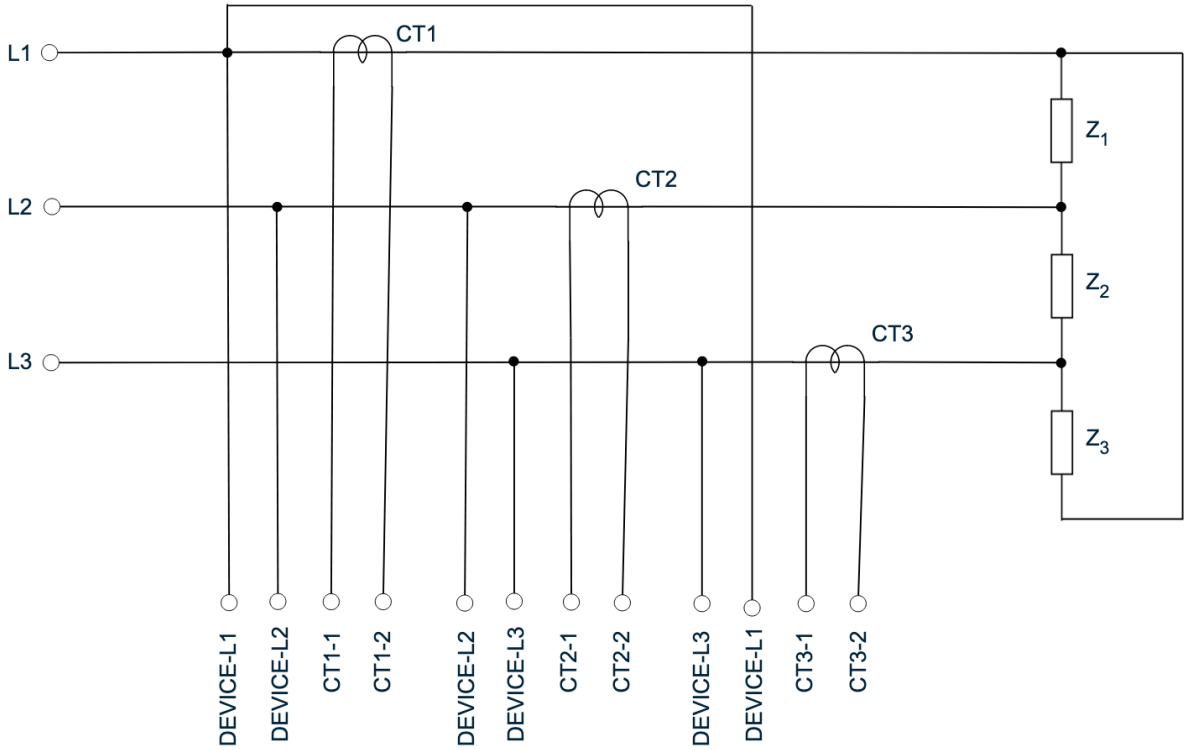

Figure 4.3: Schematic diagram of device connection in delta configuration.

```
1 pip install zeroconf
```

Figure 4.4: Python-zeroconf installation command.

```
python client_app.py
```

Figure 4.5: Starting the client application.

4.2 Manual

The manual focuses only on the client application, because after the installation of the module and it proper usage there absolutely not need to look to the device.

The client application is started with command presented in figure 4.5. Right after it is up, it starts searching with mDNS for the desired service in the local network. Each second a corresponding message is printed. When it is found the host name and port is presented to the user (figure 4.6).
Afterwards it is possible to use all measuring module functions (figure 4.7). There are twelve commands. They are selected by typing the indicated number and confirming it by pressing the enter key. Not all of them are presented below as their functionality is duplicated for each phase.

```
Welcome to the energy meter app!

Waiting for service dicover...
Service ot-service._energy_meter._udp.local. added.
Host: ot-host-F4CE3688623504BC.local
Port: 51200
```

Figure 4.6: Start of client application.

```
What do you want to do?
1. Get active energy
2. Get reactive energy
3. Get apparent energy
4. Get phase 1 rms current
5. Get phase 1 rms voltage
6. Get phase 2 rms current
7. Get phase 2 rms voltage
8. Get phase 3 rms current
9. Get phase 3 rms voltage
10. Get last boot time
11. Get last reset time
12. Reset readings
0. Exit
```

Figure 4.7: Menu of client application.

Active energy

The total active energy measured by the module is available with command 1 (figure 4.8). The value is returned in kilowatt hours [kWh].

```
Selected command: 1
Getting active energy...
Got active energy value: 0.507 [kWh]
```

Figure 4.8: Usage of command 1.

Reactive energy

The total active energy measured by the module is available with command 2 (figure 4.9). The value is returned in kilo reactive volt-amperes hours [kVARh].

```
Selected command: 2
Getting reactive energy...
Got reactive energy value: 2.733 [kVARh]
```

Figure 4.9: Usage of command 2.

Apparent energy

The total active energy measured by the module is available with command 3 (figure 4.10). The value is returned in kilo volt-amperes hours [kVAh].

```
Selected command: 3
Getting apparent energy...
Got apparent energy value: 2.804 [kVAh]
```

Figure 4.10: Usage of command 3.

Current RMS

The actual reading of current RMS value are available for each phase separately, with command 4, 6 or 8 (figure 4.11) for phase 1, 2 or 3 respectively. The values are returned in
amperes [A].

```
Selected command: 4
Getting phase 1 rms current...
Got phase 1 rms current value: 2.134 [A]
```

Figure 4.11: Usage of command 4, 6 and 8.

Voltage RMS

The actual reading of voltage RMS value are available for each phase separately, with command 5, 7 or 9 (figure 4.12) for phase 1, 2 or 3 respectively. The values are returned in volts [V].

```
Selected command: 7
Getting phase 2 rms voltage...
Got phase 2 rms current value: 233.104 [V]
```

Figure 4.12: Usage of command 5, 7 and 9.

Boot time

The command 10 (figure 4.13) gives the user time and date, when the measuring module was last turned on.

```
Selected command: 10
Getting last boot time...
Got last boot time: Mon Feb 19 22:44:31 2024
```

Figure 4.13: Usage of command 10.

Reset time

The command 11 (figure 4.14) gives the user time and date, when the measuring module readings, where reset.

```
Selected command: 11
Getting last reset time...
Got last reset time: Mon Feb 19 22:28:06 2024
```

Figure 4.14: Usage of command 11.

Reset

The command 12 (figure 4.15) resets the accumulated values of active, reactive and apparent energies.

```
Selected command: 12
Reseting readings...
Reset module readings.
```

Figure 4.15: Usage of command 12.

Exit

The command 0 simply turn off the client application.

Unknown command

In the event of selecting an command from outside of the defined scope a warning message is printed (figure 4.16).

```
Selected command: 13
Warning. Unknown command!
```

Figure 4.16: Usage of command 12.

The active, reactive and apparent energies values are saved in the device's memory and are restored at every startup.

4.3 Usage examples

The measurement module for three-phase electrical infrastructure may be used in three not so different configurations.
The two border cases are installing the device either right after the local power grid connection or directly to the target machine.
The third option is to place the module somewhere between the previously mentioned solutions to supervise a fraction of the electric system.

Chapter 5

Internal specification

5.1 Concept and system architecture

The system was based on a reference power meter from Analog Devices [10] and a similar project described in two articles [15] [16].
The ideas presented in those resources had been adjusted do work in three-phase installations. It was also subtly expanded by real time clock.

The system is composed of:

- microcontroller - Nordic Semiconductor nRF52840 [4] in the form of an USB dongle [3]; enables support for OpenThread protocol, SPI and I2C buses, as well as all remaining functionalities,
- metering IC - Analog Devices ADE7753 [37]; the second most important element, carries all the necessary energy calculations; this particular type is only single-phase, so three units in use,
- voltage level translator - Texas Instruments TXB0108 [41]; required for the SPI, because microcontroller and metering ICs operate at different voltage levels,
- I/O expander - Texas Instruments TCA6408A [40]; two units are required to handle all not previously taken into consideration metering ICs pins, the built-in voltage level translator discards the need for additional components,
- RTC - NXP Semiconductors PCF8563 [47]; provides time and date readings, as well as an additional clock signal,
- EEPROM - STMicroelectronics M24C32 [65]; it assures non-volatile data storage.

The metering ICs are using SPI bus, while I/O expanders, RTC and EEPROM I2C bus. The voltage level translator is only enabled by one pin, with no bus operations.

5.1.1 Schematic diagram

The schematic diagram (figure 5.1 was designed according the application notes of all utilized components listed in the previous paragraph.

5.1.2 Printed circuit board layout

The PCB was designed in EAGLE as mentioned in chapter 3. As opposed to the schematic diagram the layout could not be based in detail on the documentation suggestions and recommendations, because of the limit space inside the DIN rail enclosure.

The design is presented in figure 5.2. On the underside the metering ICs with voltage transformers, the required external circuitry and wire connectors are placed. Above in the middle is the microcontroller unit, namely dongle socket, EEPROM, RTC, I/O expanders and voltage level translators. At the top the power supply unit is located.
The bottom side (figure 5.3) consist mainly of connections paths. The only exceptions are three resistor for current transformers.

The manufactured PCB is shown in figure 5.4 and figure 5.5 presents it after assembly.

5.2 Firmware

The firmware for the electronic components was written on the fundamentals of Zephyr RTOS. The main project configuration file is prj.conf, which enables all necessary option. The additional file overlay−logging.conf is used for development kit debug builds. Both .overlay files define hardware configuration. The source code for all electronic components is divided into two files, a hardware specific and a functional specific, which serve as an abstraction layer, between the used function and real hardware operation, so that the elements could be replaced with no change to the rest of code. The entire firmware architecture is illustrated in figure 5.6.

The below descriptions of hardware configuration applies to the dongle. Figure 5.7 presents redefinition of the i2c0 bus peripheral. The first part changes it's frequency and the last two changes pins ports and numbers. The same works for the SPI bus. It may occur that the desired pins are already occupied by an unused feature, in such case it maybe simply disabled to resolve the conflict (example in figure 5.8). The overlay file makes possible aliases, which improves previously mentioned abstraction. In the source code an electronic elements may be acquire by DT_ALIAS(alias) macro. Based on figure 5.9 the firmware requires for example a device with alias DT_ALIAS(rtc), but is does not have to be exactly PCF8563, as long as the other RTC supports the same set of functions.

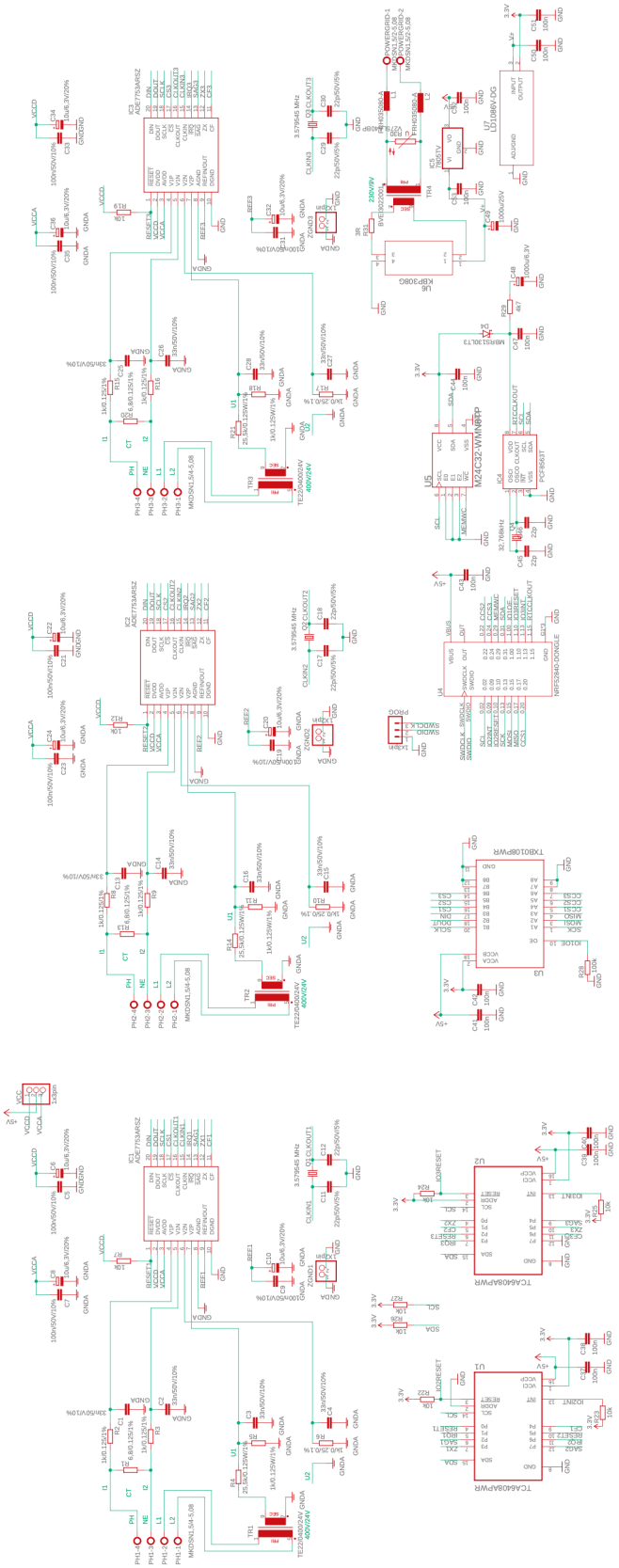

Figure 5.1: Schematic diagram of the measuring module.

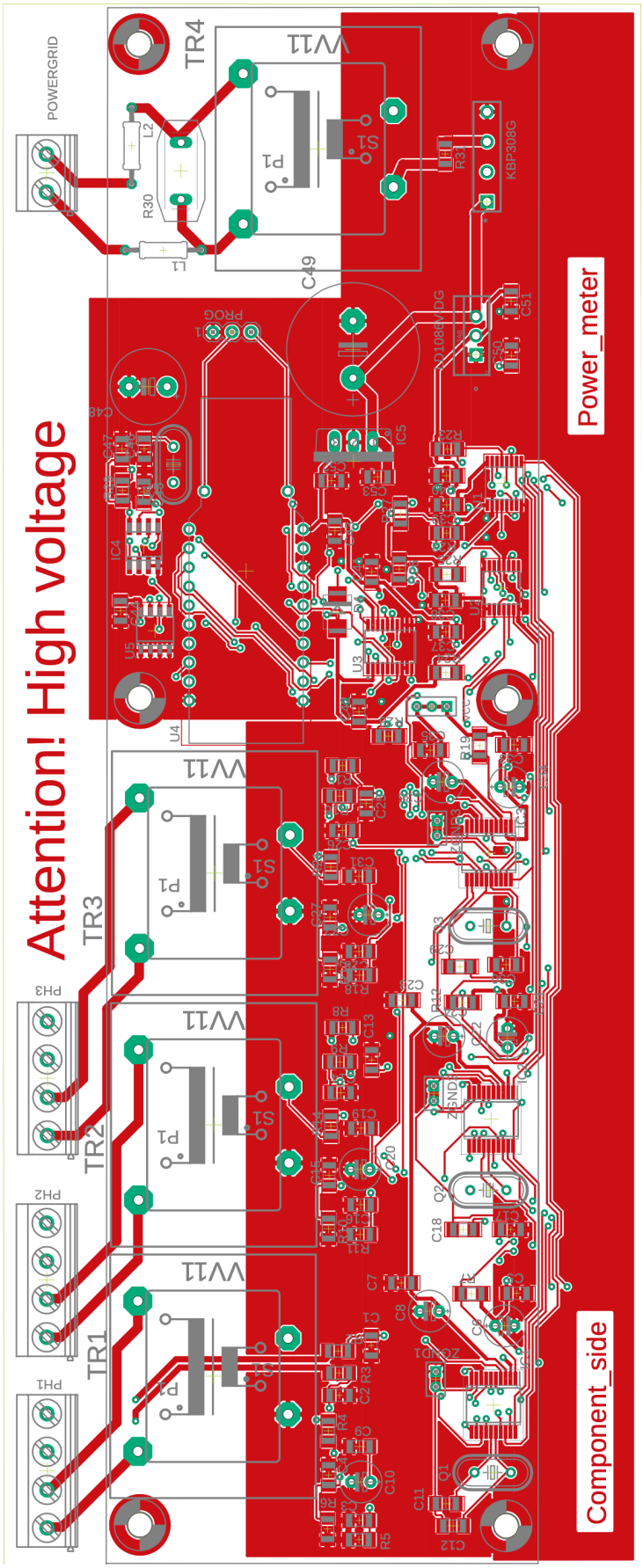

Figure 5.2: PCB top of the measuring module.

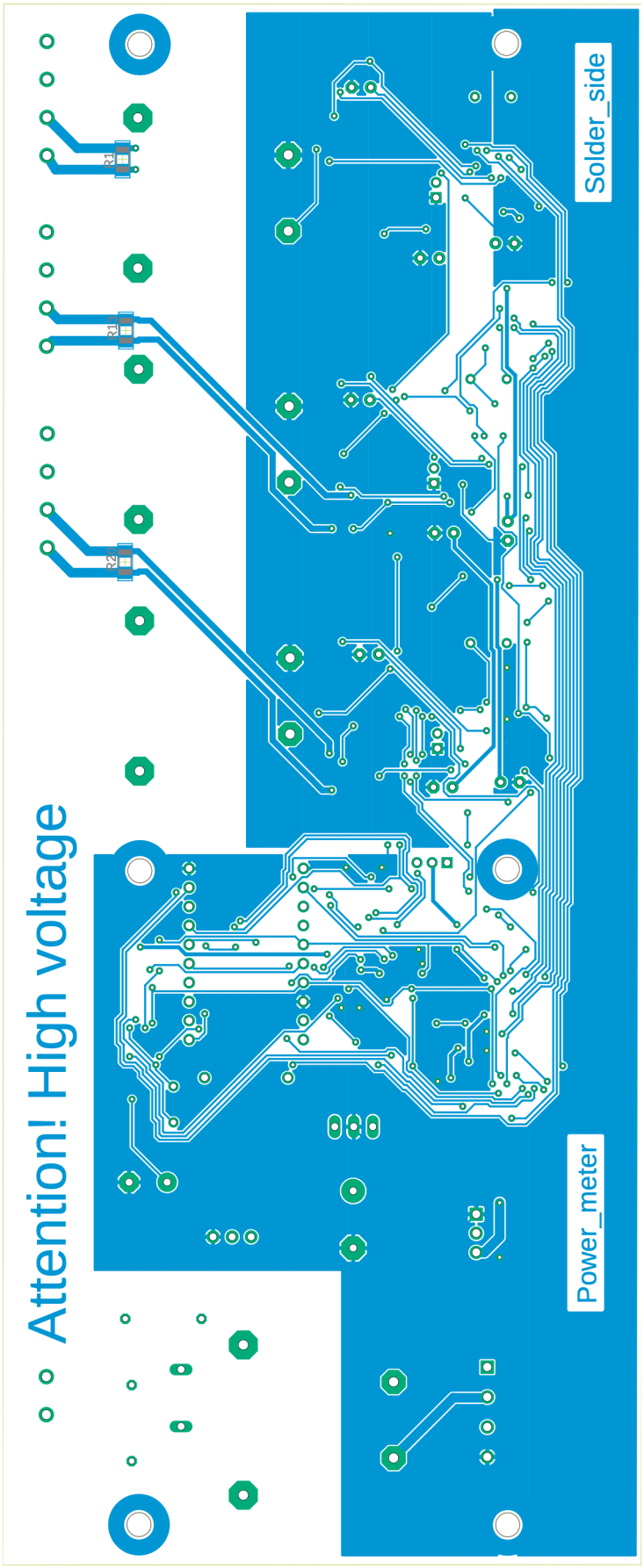

Figure 5.3: PCB bottom of the measuring module.

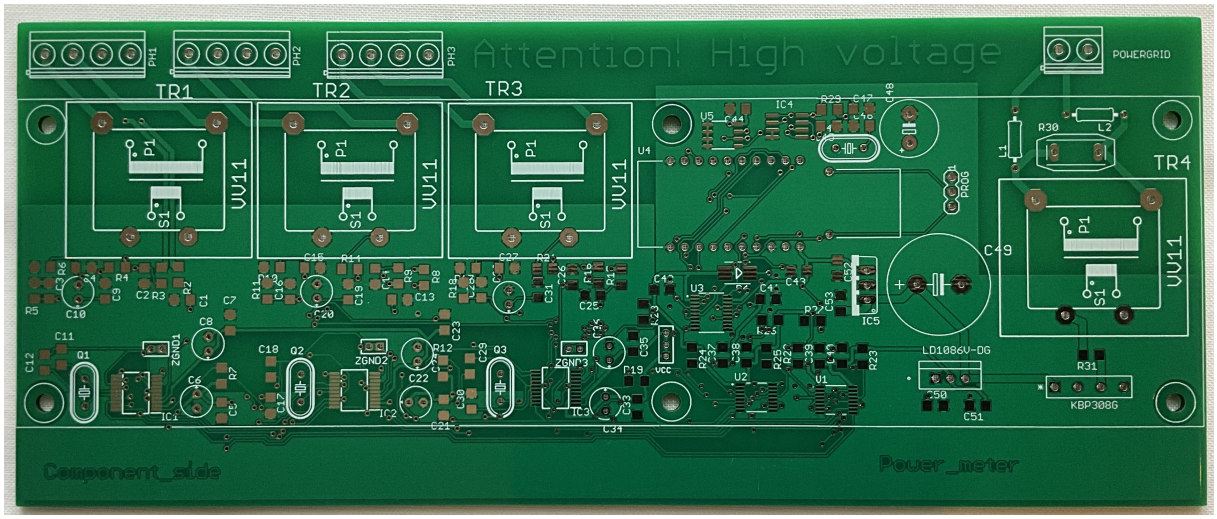

Figure 5.4: Manufactured PCB.

5.2.1 Electrically erasable programmable read-only memory

The EEPROM was the easiest component to work with. Zephyr SDK has built-in support for such kind of memory. It is enabled by the CONFIG_EEPROM switch. The hardware is defined as in figure 5.10. The compatible label defines a compatible device in Zephyr. The reg is the same as the value after @, which is device address. It can be found in the EEPROM application note [65], like all other necessary parameters. The driver API [52] includes all useful functions.

5.2.2 Real time clock

The RTC code is mostly based on the I2C example from NCS introductory course [59]. The only difference is the use of i2c_burst_read_dt() [53] function, which reads multiple number (continuous) register at once. This useful for the time and date as they are stored in 7 registers. The conversion of time is handled by the standard C Time Library time.h.

5.2.3 Input/output expander

The I/O expanders operate just on four registers and interrupt output, with standard GPIO and I2C operations. The only difficulty is to track and interpret values in those registers.

5.2.4 Metering integrated circuit

The metering ICs are the most complicated circuits in this project. Firstly, the devices have to defined inside the spi1 bus peripheral (figure 5.12). The reg is the number of Chip Select pin from figure 5.11. The data transfer is not very clear. Many buffers have to be defined (figure 5.13), a_tx_buffer and a_rx_buffer are pointers to uint8_t arrays,

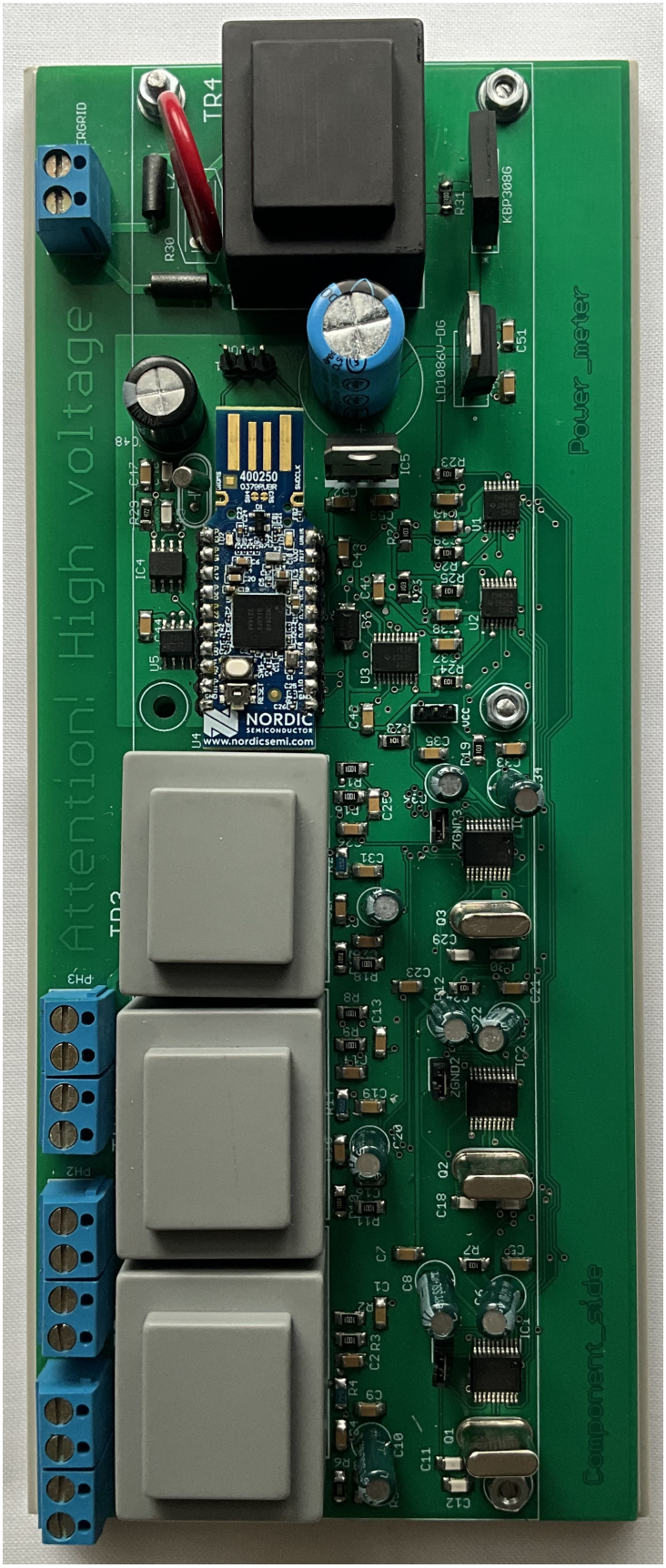

Figure 5.5: Assembled PCB.

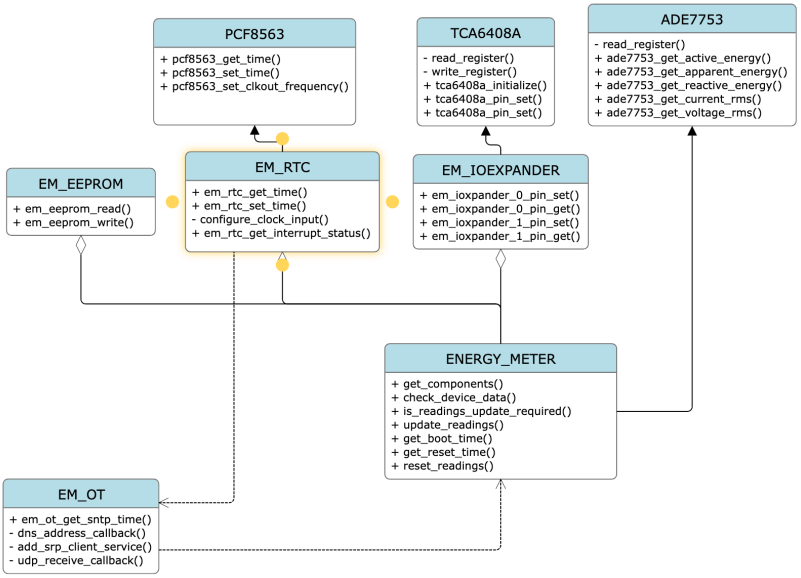

Figure 5.6: Module use case diagram.

```
&i2c0 {
    clock-frequency = < I2C_BITRATE_FAST >;
};

&i2c0_default {
    group1 {
        psels = <NRF_PSEL(TWIM_SCL, 0, 2)>,
                <NRF_PSEL(TWIM_SDA, 0, 31)>;
    };
};

&i2c0_sleep {
    group1 {
        psels = <NRF_PSEL(TWIM_SCL, 0, 2)>,
                <NRF_PSEL(TWIM_SDA, 0, 31)>;
    };
};
```

Figure 5.7: I2C bus redefinition.

```
&uart0 {
    status = "disabled";
};
```

Figure 5.8: Disabling the conflicting UART port.

```
/ {
    aliases {
    eeprom = &m24c32;
    rtc = &pcf8563;
    ioexpander0 = &tca6408a_0;
    ioexpander1 = &tca6408a_1;
    energymeter0 = &ade7753_0;
    energymeter1 = &ade7753_1;
    energymeter2 = &ade7753_2;
    };
};
```

Figure 5.9: Using aliases in overlay files.

```
m24c32: m24c32@50 {
        compatible = "atmel,at24";
        status = "okay";
        reg = < 0x50 >;
        size = < 4096 >;
        pagesize = < 32 >;
        address-width = < 16 >;
        timeout = < 5 >;
        wp-gpios = < &gpio0 29 GPIO_PULL_UP >;
    };
```

Figure 5.10: EEPROM overlay definition.

```
cs-gpios = < &gpio0 20 GPIO_ACTIVE_LOW >,
           < &gpio0 22 GPIO_ACTIVE_LOW >,
           < &gpio0 24 GPIO_ACTIVE_LOW >;
```

Figure 5.11: SPI bus devices Chip Select pins definition.

```
ade7753_0: ade7753@0 {
        compatible = "spi-device";
        reg = < 0 >;
        spi-max-frequency = < 10000000 >;
        label = "Energy Meter 0";
    };
```

Figure 5.12: SPI bus device definition.

a_tx_buffer_length and a_rx_buffer_length are their respective lengths. The read from register is started by enabling (low voltage level) the CS pin. Afterwards the investigated register number is written to the device (spi_write_dt) and data searched data is read from bus (spi_read_dt). Finally, the CS pin is disabled (high voltage level). The entire documentation is available at [55].

The flowchart in figure 5.14 presents, how readings are gathered from all three metering ICs and later proceed.

5.3 OpenThread

The base for the OpenThread part was a CLI sample from Nordic Semiconductor included included in the Zephyr SDK. This part is contained in em_ot.c source file. To get access to the OT functionality the header file zephyr/net/openthread.h has to be included. It contains two most important functions: **struct** openthread_context *openthread_get_default_context (**void**) and **struct** otInstance *openthread_get_default_instance(**void**). The default context and instance are required by most other functions from the OT library. They are stored as static global variables s_context and s_instance respectively to reduce the access overhead. The function **int** openthread_start(**struct** openthread_context *ot_context) starts OpenThread network on the device.

All the functionalities presented in the further subsections are enabled by the CONFIG_OPENTHREAD_NORDIC_LIBRARY_FTD Kconfig option, which defines the Full Thread Device.

```
const struct spi_buf tx_spi_buf[] = {
        { .buf = a_tx_buffer, .len = a_tx_buffer_length }
    };

    const struct spi_buf_set tx_spi_buf_set = {
        .buffers = tx_spi_buf,
        .count = 1
    };

    const struct spi_buf rx_spi_buf[] = {
        { .buf = a_rx_buffer, .len = a_rx_buffer_length }
    };

    const struct spi_buf_set rx_spi_buf_set = {
        .buffers = rx_spi_buf,
        .count = 1
    };
```

Figure 5.13: SPI data buffers definitions.

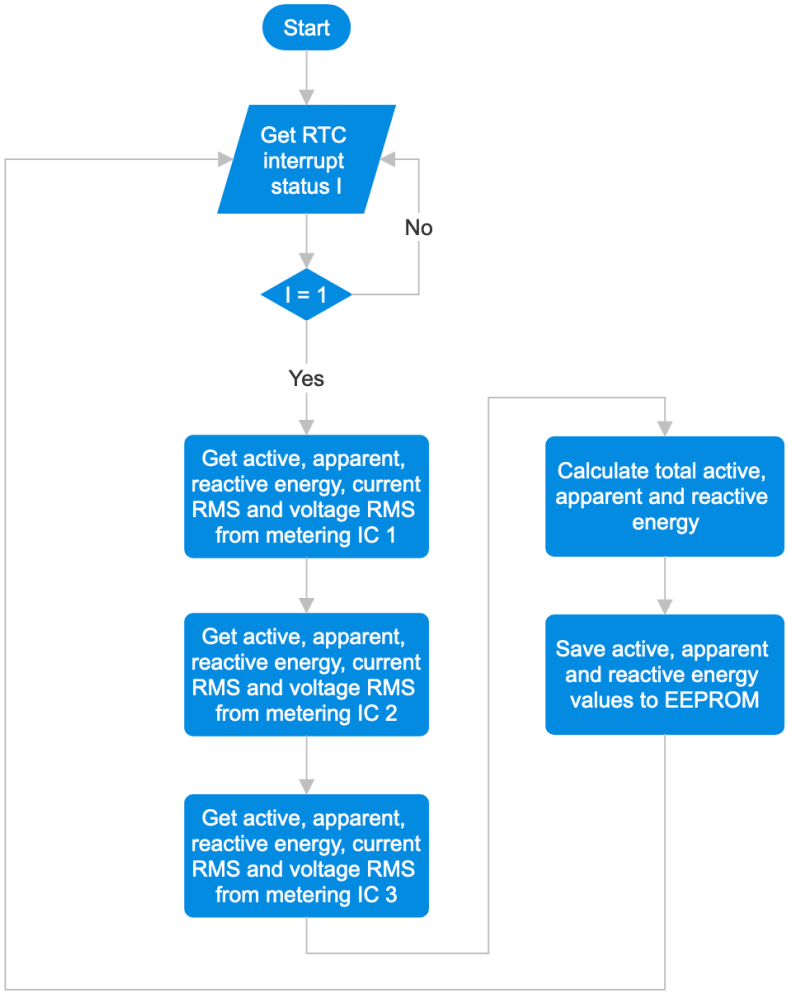

Figure 5.14: Flowchart of data gathering from metering ICs.

```
1 CONFIG_OPENTHREAD_JOINER_AUTOSTART=y
2 CONFIG_OPENTHREAD_JOINER_PSKD="M3T3R1T"
```

Figure 5.15: Joiner settings.

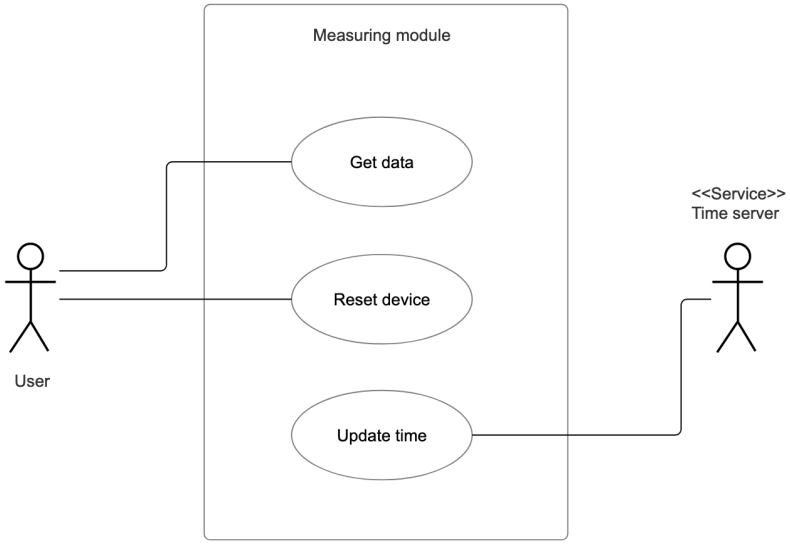

Figure 5.16: Module use case diagram.

One of them is the Joiner. It may be also enabled manually with OPENTHREAD_JOINER. Figure 5.15 presents additional settings, like the auto start, which starts the Joiner every time the module is powered on and the definition of Pre-Shared Key for the Joiner (PSKd). They are utilized by the commissioning - adding the measuring module to the Thread network.

The use case diagram in figure 5.16 illustrates, how OT exposes the metering module functionalities to the user and other services.

5.3.1 Error

The openthread/error.h [19] header file constrains only two things. The **enum** otError, which describes all possible results of most OpenThread functions. The **const char** * otThreadErrorToString(otError aError) function returns a human readable string from the given error enumeration. It is used for all compatible functions.

5.3.2 Instance and Thread

The openthread/thread.h [20] header file is needed by the otStateChangedCallback function 1 from openthread/instance.h [21] to properly identify the current device role with otThreadGetDeviceRole() and signalize it with a LED. It is similar to the OT API example

```
#define SRP_CLIENT_HOST_NAME "ot-host"

#define UDP_SRP_CLIENT_SERVICE_INSTANCE_NAME "ot-service"
#define UDP_SRP_CLIENT_SERVICE_NAME "_energy_meter._udp"
#define UDP_SRP_CLIENT_SERVICE_PORT 51200
```

Figure 5.17: Definitions for SRP client service.

[17]. The otChangedFlags indicates more state / configuration changes. The callback is set, with otError otSetStateChangedCallback(otInstance *aInstance, otStateChangedCallback aCallback, **void** *aContext) before openthread_start() call.

5.3.3 Service registration protocol

The SRP is a very useful protocol. The client application must know only the service name (defined in 5.17) to get the host name and port required to establish the communication between the module and app. The idea behind this part is based on a chapter from the Border Router guide [31]. It is enabled by the FTD option or manually with the OPENTHREAD_SRP_CLIENT Kconfig option. The header file openthread/srp_client.h [30] defines all the necessary functions.

The SRP client is initialized by three functions. Firstly, otError otSrpClientEnableAutoHostAddress(otInstance *aInstance) sets the host address to be the same as Thread network interface address.
Secondly, **void** otSrpClientEnableAutoStartMode(otInstance *aInstance, otSrpClientAutoStartCallback aCallback, **void** *aContext) monitors network data to automatically start and stop the client. The callback to that function provides only information on the start or stop and SRP server address and port.
Thirdly, otError otSrpClientSetHostName(otInstance *aInstance, **const char** *aName) sets the host name label. String pointed by aName must persist and stay unchanged after returning from this function. Before calling this last function the factory-assigned EUI-64 number (obtained with **void** otPlatRadioGetIeeeEui64(otInstance *instance, uint8_t * ieee_eui64)) may be added to the host name to uniquely identify the device among others in the local network.

Figure 5.18 shows how to add a service. It is important to mention that otSrpClientService srp_client_service is static, because it has to persist and remain unchanged after returning from this function.

```
1  static otError add_srp_client_service(void)
2  {
3      LOG_MODULE_DECLARE(OT_LOG_PROJECT_NAME, OT_LOG_LEVEL);
4
5      static otSrpClientService srp_client_service;
6      memset(&srp_client_service, 0, sizeof(srp_client_service));
7      srp_client_service.mInstanceName =
           UDP_SRP_CLIENT_SERVICE_INSTANCE_NAME;
8      srp_client_service.mName = UDP_SRP_CLIENT_SERVICE_NAME;
9      srp_client_service.mPort = UDP_SRP_CLIENT_SERVICE_PORT;
10
11     otError error = OT_ERROR_NONE;
12
13     error = otSrpClientAddService(s_instance, &
           srp_client_service);
14     if (error == OT_ERROR_NONE)
15         LOG_INF("Added␣SRP␣client␣service.");
16     else
17         LOG_ERR("Could␣not␣add␣SRP␣client␣service:␣%s!",
               otThreadErrorToString(error));
18
19     return error;
20 }
```

Figure 5.18: Function for adding a SRP client service.

5.3.4 User datagram protocol

The UDP is the base transport layer protocol used for communication between the measuring module and client application. The usage is explained in the Border Router NAT64 guide [32] No additional configuration switches are required. Only the header file openthread/udp.h [33] needs to be included.

Firstly, the UDP socket must be initialized (figure 2). The structures **static** otUdpSocket s_udp_socket (global variable, to be used later by the callback) and otSockAddr udp_socket_address must be initialized. The second variable stores port number (the same as in figure 5.17). Then the socket is opened with otError otUdpOpen(otInstance *aInstance , otUdpSocket *aSocket, otUdpReceive aCallback, **void** *aContext) and bound by otError otUdpBind(otInstance *aInstance, otUdpSocket *aSocket, **const** otSockAddr *aSockName, otNetifIdentifier aNetif).

Afterwards, the callback function **typedef void** (*otUdpReceive)(**void** *aContext, otMessage *aMessage, **const** otMessageInfo *aMessageInfo) is defined. It will not be presented entirely as it comprises more than 200 lines of code. Only the most important parts will be stated.
At first the incoming otMessage *aMessage must be read (figure 3). It is done with uint16_t otMessageRead(**const** otMessage *aMessage, uint16_t aOffset, **void** *aBuf, uint16_t aLength). The aOffset and aLength can be easily obtained with uint16_t otMessageGetOffset(**const** otMessage *aMessage) and uint16_t otMessageGetLength(**const** otMessage *aMessage) respectively. The aBuf may be for example an uint8_t array. In this project's the message incoming to the device is just one byte, which indicates the desired function.
Next the response message is created, otMessage *otUdpNewMessage(otInstance *aInstance , **const** otMessageSettings *aSettings). For default settings aSettings should be NULL. The message is composed by otError otMessageAppend(otMessage *aMessage, **const void** *aBuf, uint16_t aLength). It may be used several times for different types of aBuf content, like **double** or int64_t (the selection of response data is partially presented in figure 4, while the data appending in figure 5). In case of any error, the allocated message buffer must be freed, **void** otMessageFree(otMessage *aMessage).
Finally, when the message is complete, it is send with otError otUdpSend(otInstance * aInstance, otUdpSocket *aSocket, otMessage *aMessage, **const** otMessageInfo *aMessageInfo). Only the aMessageInfo has not been discussed. The simplest solution is to pass the aMessageInfo from the callback definition.

5.3.5 Domain name system

The DNS helps to fix the problem with the SNTP server address mentioned in the next subsection. The functionality is presented in an CLI example [5]. It is enabled by the FTD option or manually with the OPENTHREAD_DNS_CLIENT Kconfig option. The header file openthread/dns_client.h [18] includes many functions, but only three are needed. The resolve is performed by the otError otDnsClientResolveIp4Address(otInstance *aInstance, **const char** *aHostName, otDnsAddressCallback aCallback, **void** *aContext, **const** otDnsQueryConfig *aConfig) function. It may be confusing that function name indicates a IPv4 address, but the format is adjusted to be a correct IPv6 address. Host name aHostName is defined simply as "time.google.com". The **const** otDnsQueryConfig *aConfig may be NULL, so the default configuration will be used. The last issue is to define the callback function 6. The searched address is obtained with otError otDnsAddressResponseGetAddress(**const** otDnsAddressResponse *aResponse, uint16_t aIndex, otIp6Address *aAddress, uint32_t *aTtl) function. The aResponse is given directly in the callback. DNS resolve may contain more then one address and it is selected by the aIndex variable. The output of address aAddress points to the global **static** otIp6Address s_sntp_address variable, so it can further used by SNTP. The openthread/ip6.h [22] defines a function **void** otIp6AddressToString(**const** otIp6Address *aAddress, **char** *aBuffer, uint16_t aSize) to get the IPv6 address in human readable form and the recommended size for string representation of an IPv6 address OT_IP6_ADDRESS_STRING_SIZE.

The DNS utilizes semaphore mechanism from Zephyr RTOS [54]. The result of DNS resolve is returned asynchronously, so the required address may be needed before it is received. The semaphore stops program execution, while waiting for data. It is defined by K_SEM_DEFINE(dns_lock, 0, 1). The first parameter is name, second initial count and third count limit. The program runs, when count is under it's limit. Count is increased with k_sem_take(&dns_lock, K_FOREVER) (second parameter defines the time of which the increase is held) and decreased with k_sem_give(&dns_lock).

5.3.6 Simple network time protocol

The SNTP is used to get the actual time from a server, so the user does not have to enter it manually. The functionality is presented in an CLI example [6]. It is enabled by the FTD option or manually with the OPENTHREAD_SNTP_CLIENT Kconfig option. The openthread/sntp.h [29] header file contains only a few elements. Firstly the query structure needs to be defined (figure 5.19). The header file defines a default server address **#define** OT_SNTP_DEFAULT_SERVER_IP "2001:4860:4806:8::", unfortunately there were some issues with it and the query uses an DNS resolved address. The port remains

```
otMessageInfo message_info;
memset(&message_info, 0, sizeof(message_info));
message_info.mPeerPort = OT_SNTP_DEFAULT_SERVER_PORT;
message_info.mPeerAddr = s_sntp_address;

otSntpQuery sntp_query;
sntp_query.mMessageInfo = &message_info;
```

Figure 5.19: Initialization on SNTP query.

the same, as the new resulting address points the same server. Lastly, a handler function is defined. In the type definition **typedef void** (∗otSntpResponseHandler)(**void** ∗aContext, uint64_t aTime, otError aResult) it can observed that the required Unix epoch is given directly in the function, as the second parameter. If there is no error, it is assigned to a global variable for easier access. Like in DNS resolving case a semaphore ensures that the response arrives, before the time read. The query is send by otError otSntpClientQuery(otInstance ∗ aInstance, **const** otSntpQuery ∗aQuery, otSntpResponseHandler aHandler, **void** ∗aContext) function.

5.4 Command line application

The command line application is written in Python 3.9.6. The most important module is the zeroconf [14] package used for the mDNS functionality. It finds the measuring device in the local area network, by knowing only the service name - "_energy_meter._udp.local.". The included **class** ServiceListener has to be reimplemented. Mostly used is the add_service() method (figure 5.20). If the service is found and added, the associated host and port number are stored in global variables. Figure 5.21 presents how to start the service browser. The service variable is the name of searched service. The device_found variable stops the execution of the program till the browser realizes it's purpose. After acquiring the host name and port number, the usual socket operations are used to send and receive packets (figure 5.22). Even though the module operates in an OpenThread network the base IPv6 technology makes no difference between OT and standard Wi-Fi transportation layer protocol communication. The command is a 1 byte value, which tells the device, what function is requested. After the feedback the obtained bytes are transformed with the **struct**.unpack() [13] method to a floating point number. In case of time represented by a Unix epoch (amount of seconds since 00:00:00 01-01-1970), the unpack method has a different format (figure 5.23) for integer type and additionally uses time.ctime() method to get the time in Python-compatible format.

```
def add_service(self, zc: Zeroconf, type_: str, name: str) ->
    None:
    print(f"Service {name} added.")

    info = zc.get_service_info(type_, name)

    global host
    host = info.server[:len(info.server)-1]
    print(f"Host: {host}")

    global port
    port = info.port
    print(f"Port: {port}")

    global device_found
    device_found = True
```

Figure 5.20: Redefinition of add_service() method from ServiceListener class.

```
zeroconf = Zeroconf()
listener = Listener()
browser = ServiceBrowser(zeroconf, service, listener)

while device_found == False:
    print("Waiting for service dicover...")
    time.sleep(1.0)

browser.cancel()
```

Figure 5.21: Browsing for service.

```
def udp_transceive_float(command: bytes):
    try:
        udp_socket.sendto(command, (host, port))
    except:
        print(f"Could not send message to the host: {host} and
            port: {port}")

    try:
        server_response = udp_socket.recvfrom(buffer_size)[0]
    except:
        print(f"Could not recive message from the host: {host}
            and port: {port}")

    if server_response[0:1] == command:
        value = struct.unpack('d', server_response[1:])[0]
        return value
    else:
        print("Error. Invalid response!")
        raise ValueError
```

Figure 5.22: Sending a command to the measuring module and receiving a floating-point number.

```
t = time.ctime(struct.unpack('q', server_response[1:])[0])
```

Figure 5.23: Time from Unix epoch bytes.

Chapter 6

Verification and validation

6.1 Testing

Hardware

After the electronics's assembly the soldering quality was investigated with a magnifier. Then the module was powered and correctness of voltage values was examined. Additionally during the project's development some connections were tested for continuity. Actual hardware error, discussed in the next section, were catch during the firmware creation.

Firmware

The firmware testing was as complex as it's writing. The tests were performed manually not automatically. The validation was based mostly on logging messages with the nRF52840 development kit and it's real-time terminal feature. Majority of the used functions return a error value, which is thereafter expanded into status information printed to the terminal. Additionally variable values may be added for debugging purpose.

Software

The client application required the least effort. Python belongs to the most straightforward programming languages and only minor mistakes, such as variables scope and data access in array occurred, which were immediately fixed.

6.2 Detected and fixed bugs

Voltage level translator

The voltage level translator TXB0108 was chosen basing on it's documentation, which ensured the desired parameters, like number of pins, operational voltage, data direction

and rate. Unfortunately, when verifying it's operation with an oscilloscope the clock signal was not as expected. The IC was driven with a test signal of 16 clock pulses. The voltage levels and pulse duration were correct, but their number was rarely equal to 16, either less like 8 and 12 or more like 20 (it was maximum number of pulses visible on the oscilloscope screen, there could be more).
There a few solutions to this malfunction. Firstly it was decided to validate, if they are working aright. Many solutions were took into account, like a similar IC TXB0104 designed for the SPI bus, single transistor circuits and fixed-direction or direction controlled translators and shifters (e.g SN74LVC2T45). Among them only the last one gave promising outcomes, nonetheless this let to an problem with the incompatible footprint and connections, so the element could not be directly replaced.
When investigating the ADE7753 application note, the SPI input pins DIN, SCLK and $\overline{CS}$ for logic high state require at least 2.4 [V], so the nRF52840 with 3.3 [V] logic is enough to drive the bus inputs and the respective translator pads on the PCB may be shorted. In case of the DOUT output pin, the minimal 4 [V] for logic "1" value would damage the microcontroller, therefore some kind voltage change is required.
The SN74LV1T34 [39] is a single channel logic level shifter selected to resolve this last issue. Is the most simplest IC. It has only 4 connections: power, ground, input and output. The power has the same voltage as desired for output level, in this case 3.3 [V]. It is mounted on a PCB adapter with correct soldering pads and outbound 2.54 [mm] raster pins. That adapter is glued on the module's PCB and connected with wires to appropriate pads prepared for the TXB0108 (figure 6.1).

Metering IC - channel 2 voltage divider

During the design of the metering module it was supposed that the voltage transducer for the metering ICs will be a transformer with output voltage of 12 [V]. Notwithstanding, such part was out of stock and instead transformers with output of 24 [V] were ordered. However, the voltage divider used to lower the signal amplitude was not recalculated for the twice higher input value. Thankfully, when the problem was noticed only three small changes were necessary, one for each phase. The resistors R4, R14 and R21 with value of 25.5 [kΩ] had been replaced with 75 [kΩ].

6.3 Measurement test

On the grounds of time constraints and limited access to referential metrological equipment only one full measurement test could be performed. The test set is presented in figure 6.2. The module was connected to two engines connected in parallel in the 4-wire Wye three-phase configuration. They were running for about 30 minutes. The conclusions are

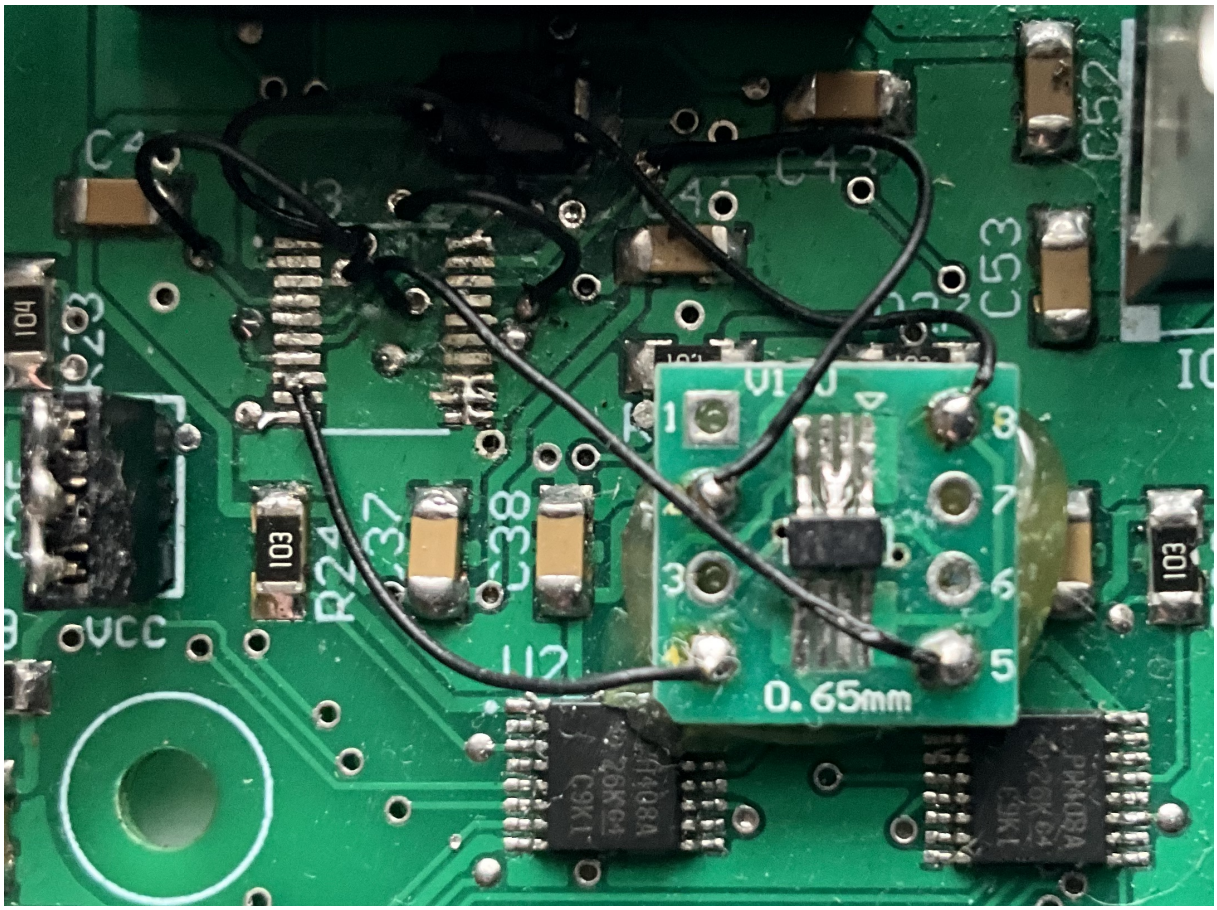

Figure 6.1: Detailed presentation of SN74LV1T34 mounting.

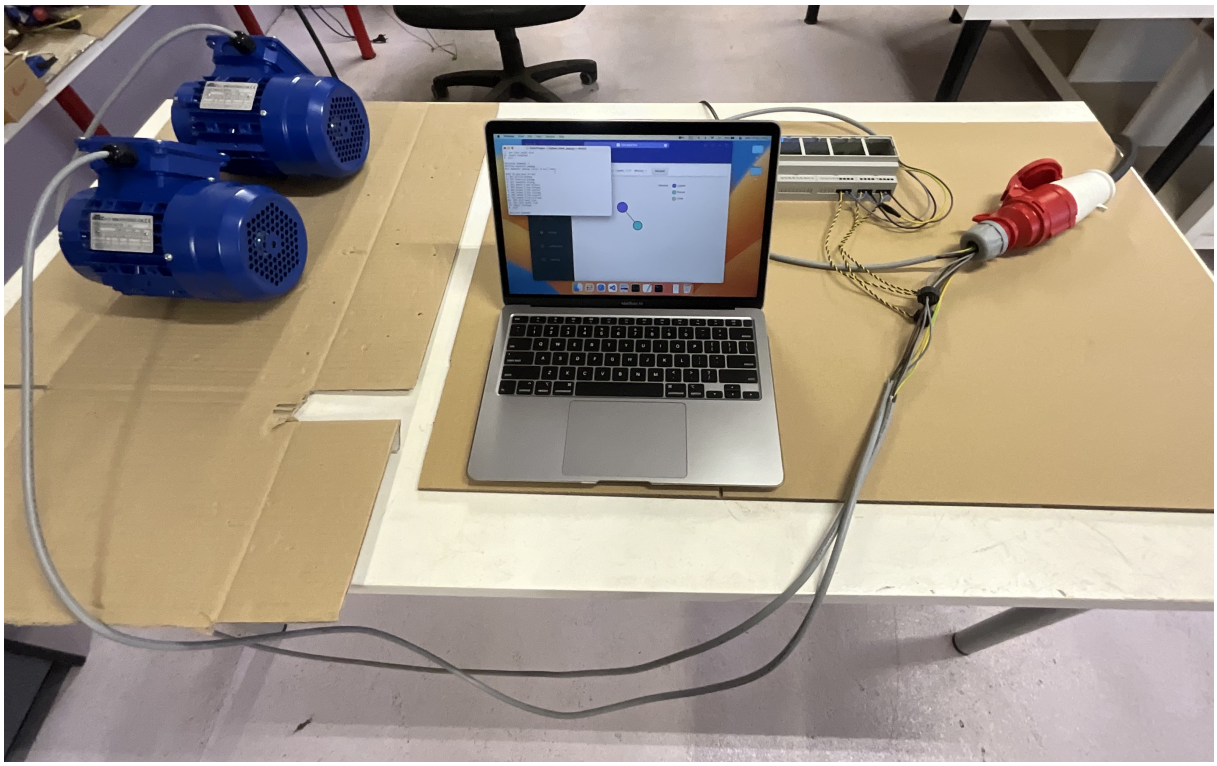

Figure 6.2: Measuring module test set.

discussed in the next section.

6.4 Result verification

The obtained measurement results presented in table 6.1. The expected current, active, reactive and apparent energies were derived from data from the engines type plates. The expected voltage was measured by a multimeter. The exceed from specified 230 [V] is not unusual, as the power grid parameters may vary depending on location and time.
The measured values from the table's lower part are within the acceptable tolerance range. In the upper part only the active energy reading is admissible. Both reactive and apparent energy overrun their expected values by more than 400 [%]

The measuring ICs, mostly one, were tested separately during project's development with typical 2200 [W] hairdryer. No problems were caught, nonetheless seeing such improper results, that test was repeated with greater attention. The first IC readings were consistent. For voltage of 218.688 [V], current of 11.584 [A] and a 10 minutes run the energies were accumulated as follows, active 0.365 [kWh], reactive 0.232 [kVARh] and 0.440 [kVAh], while the theoretical values are respectively 0.366 [kWh], 0.207 [kVARh] and 0.421 [kVAh]. Both second and third IC passed the active energy measurement, but failed reactive and apparent measurements. Based on this knowledge a conclusion can be made that those two ICs are malfunctioning and they should be replaced.

Table 6.1: Measured and expected values

Quantity	**Expected value**	**Measured value**
Active energy [kWh]	0.638	0.672
Reactive energy [kVARh]	0.357	3.619
Apparent energy [kVAh]	0.731	3.704
Current RMS - Phase 1 [A]	2.12	2.145
Voltage RMS - Phase 1 [V]	230	227.049
Current RMS - Phase 2 [A]	2.12	2.181
Voltage RMS - Phase 2 [V]	232	233.104
Current RMS - Phase 3 [A]	2.12	2.211
Voltage RMS - Phase 3 [V]	237	239.078

Chapter 7

Conclusions

7.1 Objectives and requirements evaluation

The main and only objective of this thesis is the design and construction of a measurement module for three-phase electrical infrastructure. This goal was achieved. More specific requirements were described in chapter 3. They were verified in the thesis's body. Below is presented a short summary.

Functional requirements

1. measuring module shall operate in 3-phase installations - **requirement fulfilled completely**,
2. must enable the measurement of active, reactive and apparent energy - **requirement fulfilled partially**,
3. should have a DIN rail enclosure - **requirement fulfilled completely**,
4. shall use OpenThread protocol - **requirement fulfilled completely**,
5. shall be powered directly from the power grid without any adapter - **requirement fulfilled completely**,
6. client device shall communicate with the module, while connected only to standard Wi-Fi / Ethernet network - **requirement fulfilled completely**.

Nonfunctional requirements

1. the presence of high voltage and current shall be limited - **requirement fulfilled completely**,
2. the number of external connections shall be reduced to minimum - **requirement fulfilled partially**,

3. the installation process shall be simple, only one Flathead screwdriver needed - **requirement fulfilled partially**,
4. the device can be added to a network with use of simple credentials - **requirement fulfilled completely**,
5. the user's software shall be easy to use, based on straightforward commands - **requirement fulfilled completely**,
6. the calibration parameters shall be calculated mostly by the module itself - **requirement not fulfilled**.

7.2 Further development and improvements

During the project's design and development some features had to be discarded, mostly because of limited time and electronics shortage. Perhaps these concepts presented below will be included in the next iteration or even iterations.

Utilizing all ADE7753 features

Not all of the available features were possible to implement during the project's development, like the zero crossing, channels peak level and SPI bus checksum. Moreover the calibration possibilities have not been touched at all.

Zephyr driver for RTC and I/O expander

The code for both components could be transitioned to the Zephyr Driver API. This action requires more research and testing, however it would simplify the usage of those elements. It would be also a helpful contribution to the Zephyr community.

Graphical user interface

A GUI for the client application would significantly ease the usage of the metering module and improve the presentation of the obtained data. It could even evolve into a mobile application as there are no further contraindications.

Matter

Matter [1] is a new connectivity standard developed by the Connectivity Standards Alliance (formerly the Zigbee Alliance) and first published in October 2022. It supports Thread, Wi-Fi and Bluetooth Low Energy. Most IoT devices on the market use Thread only via Matter, like the Apple HomePod mini, which serves as a border router. The metering module sees the networks propagated by those devices, however without Matter

it cannot join them. It would make also possible to use a smartphone as an external commissioner with device specific QR code to connect to the OpenThread network instead of the predefined joiner credential.

Energy price calculation

There could be an option to set a fixed energy cost to calculate the future energy bill. With the Internet connection, it could also used to calculate the cost based on dynamical prices. The data could be obtained for example from the Towarowa Giełda Energii.

Zone metering

With the current design, there are three seperate metering ICs, which may be used for changing the meter from three-phase to three-zone one-phase. For example to measure the usage of individual machines or rooms. It would also possible to add a power switch option to each line.

More capable microcontroller

The nRF52840 is best microcontroller in the 52 Series, nevertheless Nordic Semiconductor has more advanced series. The 53 Series offers one unit. It is the nRF5340 [62], a dual-core microcontroller. One core can be used for network, while the other for application, like in the network co-processor design. The brand new 54 Series for now is only announced [58]. The nRF54H20 is supposed to have very interesting RISC-V [42] architecture co-processors. In order to change the microcontroller, probably the dongle connector should replaced with solder pads. As well some additional LEDs may be included for signaling the system status and operation.

RF front-end module

The front-end module is basically a radio signal amplifier, it extends the network range and increases connection quality. Nordic Seminconductor offers one FEM, the nRF21540 [60]. It is compatible with both nRF52840 and nRF5340. However the HomePod mini design [8] shows that the SoC and FEM do not have come from the same manufacturer as it uses a Nordic Semiconductor nRF52833 with Skyworks SKY66404-11 Front-End Module.

RTC interrupt output

The interrupt output is unused, because of lack of GPIO pins. One I/O expander has free pins, but it has higher voltage. It could be used as a notificator of time periods for different electrical energy price.

Redesigned power supply unit

Changing the power supply unit type from linear to a switched-mode with SMD elements, would reduce overall size of this component.

Three-phase metering integrated circuit

On the grounds of the end electronics shortage the three-phase metering IC was unable to buy. Such IC would greatly simplify the electronic and firmware design.

Different current and voltage transducers

The current and voltage transformers have a very important property, the galvanic isolation from the power grid, which increases the safety level. However if there would be a need to reduce the module size, additional options should be taken into account. Probably the shunt resistor for current channel and direct connection for voltage channel are most considered especially, when a single three-phase metering IC would be used, where the isolation could be easily moved between the IC MCU.

External data storage

During the development of this project, it was noticed that the EEPROM capacity, even that is not among the smallest, cannot store large amount of data. Basing on the calculations, if the readings data is saved every second, the storage will last only for about 3 minutes. A SD or micro SD card would be excellent addition. Zephyr SDK has support for such solution.

Display with touch control

Such display would be a nice addition for the end user, as he would be able make use of the metering module without any external device. It was planned for a long time, however the search for a appropriate component was unfortunately unsuccessful and the concept had to be abandoned.

Detected, but not fixed bugs

The open-drain outputs $\overline{SAG}$ and $\overline{IRQ}$ of the ADE7753 metering IC require 10 [kΩ] pull-up resistors, which were omitted during the design phase. The redesign of PCB does not require much time, nevertheless the whole ordering could be a little problematic. The $\overline{SAG}$ output is not important for now and the $\overline{IRQ}$ output functionality may be replaced periodic read of the interrupt status register.

7.3 Difficulties and problems

The idea behind the project is very complex. The OpenThread protocol is not well known though it has a few years. In fact the the knowledge base is limited to the OT project web page, it's GitHub repository and some Nordic Semiconductor samples. During the research and development many problems with the OT implementation have occurred. For most there was no easy answer. The source code had to be analyzed line by line and verified with the command line interface to solve the issues, which was extremely time consuming.
The electronic architecture was quite demanding. The electronics shortage had an significant impact on this project. The elements had be chosen not because they were the most suitable, but sometimes they were the only available. For example a store has listed approximately 60 different I/O expanders, but only 3 where purchasable.

Bibliography

[1] Connectivity Standards Alliance. *Build with Matter*. 2024. URL: https://csa-iot.org/all-solutions/matter/ (visited on 27/01/2024).

[2] Joshuva Arockia Dhanraj. 'Design and Development of Energy Meter for Energy Consumption'. In: Feb. 2024, pp. 563–569. ISBN: 978-981-99-6865-7.

[3] Nordic Semiconductor ASA. 'nRF52840 (PCA10059) Dongle User Guide'. In: (2018).

[4] Nordic Semiconductor ASA. 'nRF52840 Product Specification'. In: (2019).

[5] The OpenThread Authors. *GitHub - openthread/src/cli/README.md - DNS*. 2023. URL: https://github.com/openthread/openthread/blob/main/src/cli/README.md#dns-resolve4-hostname-dns-server-ip-dns-server-port-response-timeout-ms-max-tx-attempts-recursion-desired-boolean (visited on 08/02/2024).

[6] The OpenThread Authors. *GitHub - openthread/src/cli/README.md - SNTP*. 2023. URL: https://github.com/openthread/openthread/blob/main/src/cli/README.md#sntp-query-sntp-server-ip-sntp-server-port (visited on 08/02/2024).

[7] Stanisław Bolkowski. *Elektrotechnika*. Warszawa: WSiP, 1999. ISBN: 83-02-05981-1.

[8] CChin. *HomePod Mini IC Identification*. 2021. URL: https://www.ifixit.com/Guide/HomePod+Mini+IC+Identification/141641#s283808 (visited on 29/01/2024).

[9] Analog Devices. *Energy Metering ICs*. 2024. URL: https://www.analog.com/en/product-category/energy-metering-ics.html (visited on 04/02/2024).

[10] Stephen English and Dave Smith. 'A Power Meter Reference Design Based on the ADE7756'. In: (2001).

[11] Eve. *Eve Energy*. 2023. URL: https://www.evehome.com/en/eve-energy (visited on 18/02/2024).

[12] Fluke. *Fluke 1732 and 1734 Three Phase Power Measurement Logger*. 2023. URL: https://www.fluke.com/en-us/product/electrical-testing/power-quality/1732-1734 (visited on 21/02/2024).

[13] Python Software Foundation. *struct — Interpret bytes as packed binary data*. 2024. URL: https://docs.python.org/3/library/struct.html (visited on 08/02/2024).

[14] Python Software Foundation. *zeroconf PyPI*. 2024. URL: https://pypi.org/project/zeroconf/ (visited on 08/02/2024).

[15] Grzegorz Gajewski. 'Miernik energii elektrycznej i watomierz, część 1'. In: *Elektronika Praktyczna* 12 (2003), pp. 14–18.

[16] Grzegorz Gajewski. 'Miernik energii elektrycznej i watomierz, część 2'. In: *Elektronika Praktyczna* 1 (2004), pp. 49–54.

[17] Google. *Developing with OpenThread APIs*. 2023. URL: https://openthread.io/codelabs/openthread-apis (visited on 28/01/2024).

[18] Google. *DNS | OpenThread*. 2024. URL: https://openthread.io/reference/group/api-dns (visited on 17/02/2024).

[19] Google. *Error | OpenThread*. 2024. URL: https://openthread.io/reference/group/api-error (visited on 06/02/2024).

[20] Google. *General | OpenThread*. 2024. URL: https://openthread.io/reference/group/api-thread-general (visited on 07/02/2024).

[21] Google. *Instance | OpenThread*. 2024. URL: https://openthread.io/reference/group/api-instance (visited on 07/02/2024).

[22] Google. *IPv6 | OpenThread*. 2024. URL: https://openthread.io/reference/group/api-ip6 (visited on 18/02/2024).

[23] Google. *OpenThread*. 2024. URL: https://openthread.io (visited on 12/01/2024).

[24] Google. *OpenThread - Co-Processor Designs*. 2023. URL: https://openthread.io/platforms/co-processor (visited on 30/01/2024).

[25] Google. *OpenThread - IPv6 Addressing*. 2024. URL: https://openthread.io/guides/thread-primer/ipv6-addressing (visited on 21/01/2024).

[26] Google. *OpenThread - Node Roles and Types*. 2023. URL: https://openthread.io/guides/thread-primer/node-roles-and-types (visited on 30/01/2024).

[27] Google. *OpenThread Border Router*. 2024. URL: https://openthread.io/guides/border-router (visited on 28/01/2024).

[28] Google. *OpenThread C API Reference*. 2024. URL: https://openthread.io/reference (visited on 28/01/2024).

[29] Google. *SNTP | OpenThread*. 2024. URL: https://openthread.io/reference/group/api-sntp (visited on 08/02/2024).

[30] Google. *SRP | OpenThread*. 2024. URL: https://openthread.io/reference/group/api-srp (visited on 07/02/2024).

[31] Google. *Thread Border Router - Bidirectional IPv6 Connectivity and DNS-Based Service Discovery - 5. Publish the Service on the End Device*. 2023. URL: https://openthread.io/codelabs/openthread-border-router#4 (visited on 07/02/2024).

[32] Google. *Thread Border Router - Provide Internet access via NAT64 - Communicate via UDP*. 2023. URL: https://openthread.io/codelabs/openthread-border-router-nat64#communicate-via-udp (visited on 17/02/2024).

[33] Google. *UDP | OpenThread*. 2024. URL: https://openthread.io/reference/group/api-udp (visited on 17/02/2024).

[34] Google. *What is Thread?* 2023. URL: https://openthread.io/guides/thread-primer/ (visited on 30/01/2024).

[35] Shelly Group. *Shelly Pro 3EM-400*. 2023. URL: https://www.shelly.com/en-us/products/shop/shelly-pro-3-em/shelly-pro-3-em-400-a (visited on 21/02/2024).

[36] Thread Group. *Thread - Home*. 2024. URL: https://www.threadgroup.org (visited on 29/01/2024).

[37] Analog Devices Inc. 'ADE7753 Single-Phase Multifunction Metering IC with di/dt Sensor Interface'. In: (2010).

[38] Microchip Technology Inc. 'MCP3909 3-Phase Energy Meter Reference Design Using the PIC18F2520'. In: (2008).

[39] Texas Instruments Incorporated. 'SN74LV1T34 Single Power Supply Single Buffer GATE CMOS Logic Level Shifter'. In: (2023).

[40] Texas Instruments Incorporated. 'TCA6408A Low-Voltage 8-Bit I2C and SMBus I/O Expander With Interrupt Output, Reset, and Configuration Registers'. In: (2015).

[41] Texas Instruments Incorporated. 'TXB0108 8-Bit Bidirectional Voltage-Level Translator with Auto-Direction Sensing and ±15-kV ESD Protection'. In: (2020).

[42] RISC-V International. *RISC-V International*. 2023. URL: https://riscv.org (visited on 27/01/2024).

[43] Beijing Lewei IOT Technologies Co. Ltd. *Three Phase Energy Meter Wi-Fi,split phase,residential energy consumption,solar pv monitor,net energy metering, Modbus TCP/RTU*. 2023. URL: https://www.iammeter.com/products/three-phase-meter (visited on 21/02/2024).

[44] Hariharan Mani. 'Analog Devices Energy (ADE) Products: Frequently Asked Questions (FAQs)'. In: (2013).

[45] Serhiy Matviyenko. *Smart Power Monitor*. 2023. URL: https://www.hackster.io/matvs/smart-power-monitor-6f2f84 (visited on 18/02/2024).

[46] Alan Nebrida, Chavelita Amador, Clyed Madiam and Glenn Ranche. 'Development of Smart Meter to Monitor Real Time Energy Consumption for Sustainability'. In: *International Journal of Sustainable Construction Engineering and Technology* 14.10 (2023).

[47] NXP Semiconductors N.V. 'PCF8563'. In: (2015).

[48] Mateusz Pająk. *Matter Energy Harvesting Power Meter*. 2024. URL: https://www.hackster.io/matvs/smart-power-monitor-6f2f84 (visited on 18/02/2024).

[49] Józef Parchański. *Miernictwo Elektryczne i Elektroniczne*. Warszawa: WSiP, 2008. ISBN: 978-83-02-07042-6.

[50] Zephyr Project. *Configuration System (Kconfig)*. 2024. URL: https://docs.zephyrproject.org/latest/build/kconfig/index.html (visited on 18/02/2024).

[51] Zephyr Project. *Devicetree HOWTOs*. 2024. URL: https://docs.zephyrproject.org/latest/build/dts/howtos.html (visited on 18/02/2024).

[52] Zephyr Project. *Electrically Erasable Programmable Read-Only Memory (EEPROM)*. 2024. URL: https://docs.zephyrproject.org/latest/hardware/peripherals/eeprom.html (visited on 20/02/2024).

[53] Zephyr Project. *Inter-Integrated Circuit (I2C) Bus*. 2024. URL: https://docs.zephyrproject.org/latest/hardware/peripherals/i2c.html (visited on 20/02/2024).

[54] Zephyr Project. *Semaphores*. 2024. URL: https://docs.zephyrproject.org/latest/kernel/services/synchronization/semaphores.html (visited on 18/02/2024).

[55] Zephyr Project. *Serial Peripheral Interface (SPI) Bus*. 2024. URL: https://docs.zephyrproject.org/latest/hardware/peripherals/spi.html (visited on 20/02/2024).

[56] J. Woodyatt R. Quattlebaum. 'Spinel Host-Controller Protocol'. In: *Network Working Group Internet-Draft* (2017).

[57] SEGGER. *SEGGER J-Link debug probes*. 2023. URL: https://www.segger.com/products/debug-probes/j-link/ (visited on 28/01/2024).

[58] Nordic Semiconductor. *Nordic Semiconductor redefines its leadership in Bluetooth Low Energy with the announcement of the nRF54 Series*. 2024. URL: https://www.nordicsemi.com/Nordic-news/2023/04/Nordic-Semiconductor-redefines-its-leadership-in-Bluetooth-Low-Energy-with-the-nRF54-Series (visited on 27/01/2024).

[59] Nordic Semiconductor. *nRF Connect SDK Fundamentals*. 2022. URL: https://academy.nordicsemi.com/courses/nrf-connect-sdk-fundamentals/ (visited on 28/01/2024).

[60] Nordic Semiconductor. *nRF21540*. 2024. URL: https://www.nordicsemi.com/Products/nRF21540 (visited on 27/01/2024).

[61] Nordic Semiconductor. *nRF52840 DK*. 2024. URL: https://www.nordicsemi.com/Products/Development-hardware/nRF52840-DK (visited on 28/01/2024).

[62] Nordic Semiconductor. *nRF5340*. 2024. URL: https://www.nordicsemi.com/Products/nRF5340 (visited on 27/01/2024).

[63] Nordic Semiconductor. *Welcome to the nRF Connect SDK!* 2023. URL: https://developer.nordicsemi.com/nRF_Connect_SDK/doc/2.5.1/nrf/index.html (visited on 28/01/2024).

[64] NXP Semiconductors. *Three-Phase Power Meter Reference Design*. 2024. URL: https://www.nxp.com/design/design-center/designs/three-phase-power-meter-reference-design:THREE-PHASE-POWER-METER (visited on 21/02/2024).

[65] STMicroelectronics. 'M24C32'. In: (2017).

[66] STMicroelectronics. *Metering ICs*. 2024. URL: https://www.st.com/en/data-converters/metering-ics.html (visited on 04/02/2024).

[67] Microchip Technology. *Metering*. 2024. URL: https://www.microchip.com/en-us/products/smart-energy-metering/metering (visited on 04/02/2024).

[68] Guanghua Tong, Xiaohua Liu, Xinquan Li, Guodong Sun, Jian Mu and Liang Liu. 'Research on Three-Phase Electronic Multifunctional Energy Meter'. In: *OP Conference Series: Materials Science and Engineering* 394.08 (2018), p. 042098.

Appendices

Index of abbreviations and symbols

IoT	Internet of Things
DIN	Deutsches Institut für Normung e.V
EEPROM	Electrically Erasable Programmable Read-Only Memory
IC	Integrated Circuit
OT	OpenThread
SRP	Service Registration Protocol
IPv6	Internet Protocol version 6
DNS	Domain Name System
SNTP	Simple Network Time Protocol
UDP	User Datagram Protocol
DC	Direct Current
AC	Alternating Current
MAC	Medium Access Control
REED	Router Eligible End Device
SoC	System-on-Chip
RCP	Radio Co-Processor
NCP	Network Co-Processor
CLI	Command Line Interface
PCB	Printed Circuit Board
VSC	Visual Studio Code

IDE	Integrated Development Environment
GPIO	General Purpose Input/Output
NCS	nRF Connect SDK
SDK	Software Development Kit
SMD	Surface Mount Device
I2C	Inter-Integrated Circuit
SPI	Serial Peripheral Interface
OTBR	OpenThread Border Router
GUI	Graphical User Interface
mDNS	Multicast DNS
IPv4	Internet Protocol version 4
NAT64	Network Address Translation between IPv6 and IPv4
SBC	Single Board Computer
MCU	Microcontroller Unit
RTC	Real Time Clock
I/O	Input / Output
API	Application Programming Interface
RTOS	Real Time Operating System
PSKd	Pre-Shared Key for the Joiner
FEM	Front-End Module

Listings

```
1  static void state_changed_callback(otChangedFlags a_flags, void
       *a_context)
2  {
3      LOG_MODULE_DECLARE(OT_LOG_PROJECT_NAME, OT_LOG_LEVEL);
4
5      OT_UNUSED_VARIABLE(a_context);
6
7      if ((a_flags & OT_CHANGED_THREAD_ROLE) != 0)
8      {
9          otDeviceRole current_role = otThreadGetDeviceRole(
               s_instance);
10
11         switch (current_role)
12         {
13             case OT_DEVICE_ROLE_LEADER:
14             case OT_DEVICE_ROLE_ROUTER:
15             case OT_DEVICE_ROLE_CHILD:
16                 LOG_INF("Device connected to OpenThread network.
                       ");
17                 gpio_pin_set_dt(&s_state_led, 1);
18                 break;
19
20             case OT_DEVICE_ROLE_DETACHED:
21             case OT_DEVICE_ROLE_DISABLED:
22                 LOG_INF("Device not connected to OpenThread
                       network.");
23                 gpio_pin_set_dt(&s_state_led, 0);
24                 break;
25         }
26     }
27 }
```

Figure 1: OpenThread state changed callback function.

```
1  static otError initialize_udp_socket(void)
2  {
3      LOG_MODULE_DECLARE(OT_LOG_PROJECT_NAME, OT_LOG_LEVEL);
4
5      otSockAddr udp_socket_address;
6
7      memset(&s_udp_socket, 0, sizeof(s_udp_socket));
8      memset(&udp_socket_address, 0, sizeof(udp_socket_address));
9
10     udp_socket_address.mPort = UDP_SRP_CLIENT_SERVICE_PORT;
11
12     otError error = OT_ERROR_NONE;
13
14     error = otUdpOpen(s_instance, &s_udp_socket,
           udp_receive_callback, s_context);
15     if (error == OT_ERROR_NONE)
16     {
17         LOG_INF("Opened UDP socket.");
18     }
19     else
20     {
21         LOG_ERR("Could not open UDP socket: %s!",
               otThreadErrorToString(error));
22         return error;
23     }
24
25     error = otUdpBind(s_instance, &s_udp_socket, &
           udp_socket_address, OT_NETIF_THREAD);
26     if (error == OT_ERROR_NONE)
27         LOG_INF("Binded UDP socket.");
28     else
29         LOG_ERR("Could not bind UDP: %s!", otThreadErrorToString
               (error));
30
31     return error;
32 }
```

Figure 2: UDP socket initialization.

```
uint16_t message_length = otMessageGetLength(a_message);
if (message_length != 0)
{
    LOG_INF("Received UDP message of length: %i [B].",
        message_length);
}
else
{
    LOG_ERR("Received UDP message of length 0!");
    return;
}

uint16_t read_bytes = 0;

uint8_t buffer[message_length];

read_bytes = otMessageRead(a_message, otMessageGetOffset(
    a_message), buffer, message_length);
if (read_bytes == message_length)
{
    LOG_INF("Read %i [B] from UDP message.", read_bytes);
}
else
{
    LOG_ERR("Could not read UDP message!");
    return;
}
```

Figure 3: UDP callback part responsible for message read.

```
uint8_t command = 0xFF;
uint8_t *response_message;
size_t response_size = 0;

switch (buffer[0])
{
case 0x00:
    LOG_INF("0x00 command.");
    command = 0x00;
    const double active_energy = get_active_energy();
    response_size = sizeof(active_energy);
    response_message = (uint8_t*)malloc(response_size);
    memcpy(response_message, &active_energy, response_size);
    break;

    [...]

case 0x09:
    LOG_INF("0x09 command.");
    command = 0x09;
    const int64_t boot_time = get_boot_time();
    response_size = sizeof(boot_time);
    response_message = (uint8_t*)malloc(response_size);
    memcpy(response_message, &boot_time, response_size);
    break;

    [...]

case 0x0B:
    LOG_INF("0x0B command.");
    command = 0x0B;
    reset_readings();
    break;

    [...]

default:
    LOG_WRN("Unknown command!");
    response_size = sizeof("Warning. Unknown command!");
    response_message = (char*)malloc(response_size);
    strcpy(response_message, "Warning. Unknown command!");
    break;
    }
```

Figure 4: UDP callback part, piece of response data selection.

```
otError error = OT_ERROR_NONE;

error = otMessageAppend(message, &command, sizeof(command));
if (error == OT_ERROR_NONE)
{
    LOG_INF("Appended message.");
}
else
{
    LOG_ERR("Could not append message: %s!",
        otThreadErrorToString(error));
    otMessageFree(message);
    return;
}

error = otMessageAppend(message, response_message, response_size
    );
if (error == OT_ERROR_NONE)
{
    LOG_INF("Appended message.");
}
else
{
    LOG_ERR("Could not append message: %s!",
        otThreadErrorToString(error));
    otMessageFree(message);
    return;
    }
```

Figure 5: UDP callback part for appending data to the outgoing message.

```
1  static void dns_address_callback(otError a_error, const
       otDnsAddressResponse *a_response, void *a_context)
2  {
3      LOG_MODULE_DECLARE(OT_LOG_PROJECT_NAME, OT_LOG_LEVEL);
4
5      OT_UNUSED_VARIABLE(a_context);
6
7      if (a_error == OT_ERROR_NONE)
8      {
9          LOG_INF("Got DNS address response.");
10
11         a_error = otDnsAddressResponseGetAddress(a_response, 0,
               &s_sntp_address, NULL);
12
13         if (a_error == OT_ERROR_NONE)
14         {
15             otIp6Address dns_ip6_address = s_sntp_address;
16             char dns_string_address[OT_IP6_ADDRESS_STRING_SIZE];
17             otIp6AddressToString(&dns_ip6_address,
                   dns_string_address, OT_IP6_ADDRESS_STRING_SIZE);
18
19             LOG_DBG("Got address from DNS response: %s.",
                   dns_string_address);
20         }
21         else
22         {
23             LOG_ERR("Could not get address from DNS reponse: %s!
                   ", otThreadErrorToString(a_error));
24         }
25     }
26     else
27     {
28         LOG_ERR("Could not get DNS address response: %s!",
               otThreadErrorToString(a_error));
29     }
30
31     k_sem_give(&dns_lock);
32 }
```

Figure 6: DNS address callback function.

List of additional files in electronic submission

Additional files uploaded to the system include:

- EAGLE PCB design files,
- source code of the firmware,
- source code of the command line app,
- video files showing how the module is used.

List of Figures

List of Tables